University of
Southern Denmark

2nd Semester Project - HeatItOn

Heat Production Optimization

Group 8

Adela Pastekova – adpas25@student.sdu.dk
Alex Zalanyi – alzal25@student.sdu.dk
Filip Tichy – fitic25@student.sdu.dk
Marija Savkina – masav25@student.sdu.dk
Melana Ohrimeca – meohr25@student.sdu.dk
Zoltan Suveges – zosuv25@student.sdu.dk

June 2026

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Meaning
DB	Database
FE	Frontend
BE	Backend
UML	Unified Modelling Language
CRC	Class-Responsibility-Collaboration
OPT	Optimizer
DV	Data Visualization
AM	Asset Manager
SDM	Source Data Manager
RDM	Result Data Manager
UI	User Interface
UX	User Experience
WSL	Windows Subsystem for Linux
OS	Operating System
RAM	Random Access Memory
API	Application Programming Interface
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
MVVM	Model-View-ViewModel
CRUD	Create-Read-Update-Delete
SOLID	Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
DRY	Do Not Repeat Yourself
KISS	Keep It Simple, Stupid
YAGNI	You Aren't Gonna Need It
SoC	Separation of Concerns
SQL	Structured Query Language

Abstract

Efficient heat production planning is a significant challenge in modern district heating systems. In the city of Heatington, this planning is still performed manually, which makes the process time-consuming and prone to inefficiencies while supplying heat to approximately 1600 buildings. This project - HeatItOn, represents the development of a web application, providing an automated heat production platform. The project was developed using .NET C# and a MySQL database. API clients were used to provide communication between the backend and frontend components. The frontend was developed using Avalonia [1], providing a user-friendly interface with separate tabs for scenario management, source data input, and results visualization. The application consists of several modules: the Asset Manager responsible for managing assets; the Source Data Manager responsible for processing incoming data; the Optimizer responsible for the main heat production optimization logic; the Result Data Manager responsible for processing output data and the Data Visualization module responsible for presenting data through graphs and data grids. The development system enables users to input source data, select assets for optimization, and analyze the generated results through data grids and graphical visualizations. The developed system successfully automated the heat production planning workflow and generated optimized heat schedules based on user-defined scenarios. The results demonstrate that automated planning can reduce manual effort and provide clearer insights into heat production performance. Overall, the project shows that such a tool can support more efficient and data-driven decision-making in district heating operations.

Chapter 1

Introduction

1.1 Project Objectives and Requirements

The objective of the HeatItOn project is to develop an automated system that can generate cost-efficient heat production schedules for the district heating network in Heatington. The system aims to replace the current manual planning process by delivering optimized heat-production schedules that meet heat demand, reduce operational costs and ensure efficient use of energy resources. To achieve this, the system must support several key requirements: it must allow users to input relevant source data, manage and select production assets, execute an optimization algorithm that produces feasible and cost-effective schedules, process the resulting output data, and present the results through clear graphical and structured visualizations. The project was developed using a modular software engineering approach, combining .NET C#, MySQL database for data storage, and an Avalonia-based frontend. Communication between components was handled through API clients and the development process included requirement analysis, system design, implementation and testing to ensure that each module functioned reliably within the overall system.

1.2 Problem Statement

This project addressed the challenge of developing an automated heat production optimization system for HeatItOn, a centralised heating utility in the city of Heatington responsible for supplying heat to approximately 1600 buildings. The desired outcome was a working application capable of automatically generating cost-efficient heat production schedules, ensuring that the heat demand of the network was met at all times while keeping operational costs as low as possible [8].

The system had to account for different types of production units. Some units simply burn fuel to produce heat, meaning their production costs are fixed and predictable. However, other units are also connected to the electricity grid. A gas motor produces electricity as a byproduct which can be sold back to the grid, while an electric boiler consumes electricity which must be purchased. Since electricity prices change every hour, the effective cost of running these units varies constantly, requiring the optimizer to recalculate and compare the costs of all units on an hourly basis in order to always produce the most economical schedule.

This required implementing two distinct optimization scenarios. The first was a heat-only scenario using three gas boilers and one oil boiler. The second was a combined heat and electricity scenario

using two gas boilers, one gas motor, and one electric boiler. Both scenarios also required support for mandatory maintenance periods of 30 to 60 hours for designated units, during which the affected unit could not participate in the optimization.

Prior to this project, all heat production planning at HeatItOn was carried out manually. Operators had to select which production units to run in each hour based on experience and rough cost estimates. This made it particularly difficult to react to changing conditions such as fluctuating electricity prices or varying heat demand throughout the day. As a result, the process was time-consuming and prone to producing inefficient decisions. By automating this process, the aim was to make heat production planning more reliable and cost-efficient.

Problem Analysis

HeatItOn is a district heating utility in the city of Heatington, responsible for supplying heat to approximately 1,600 buildings via a centralized network. Heat is generated by a combination of traditional heat-only boilers and combined heat and power (CHP) units, which simultaneously produce heat and either generate or consume electricity [4]. In a centralized district heating system, hot water is pumped from a central production plant through underground pipes to connected buildings, where it is used for space heating and hot tap water. Once the heat has been extracted, the cooled water flows back to the plant, where it is reheated and recirculated throughout the network.

System will consist of 5 main modules:

- Optimizer (OPT)
- Data Visualization (DV)
- Asset Manager (AM)
- Source Data Manager (SDM)
- Result Data Manager (RDM)

Optimizer (OPT): Optimizer (OPT): Evaluate all available production units and calculates the most economical heat production schedule, guaranteeing that heat demand across all buildings is met at all times. Each production unit has a fixed cost expressed in DKK per MWh. The source data for this module is provided in the 2026 Heat Production Optimization – Danfoss Deliveries file.

Data Visualization (DV): Presents asset, source, and result data in a visual format, enabling users to easily read and analyze the optimization outcomes.

Asset Manager (AM): Serves as a repository for static system information, providing it available to other system modules. Through the AM, modules can retrieve heating grid information including unit names and graphical representations of the grid.

Source Data Manager (SDM): Serves as a repository for dynamic system information, storing data in local CSV files. The SDM stores and provides two types of data, the amount of heat

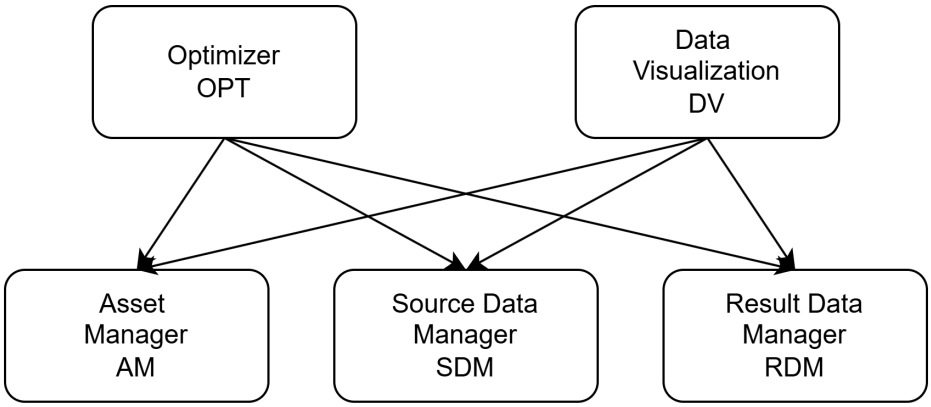


Figure 1.1: System Architecture Diagram

demanded by the buildings and the current electricity prices, both of which are updated on an hourly basis.

Result Data Manager (RDM): Scenario 1 operates on a single district heating network with three gas boilers (GB1, GB2, GB3) and one oil boiler (OB1). The gas boilers have lower production costs, while the oil boiler is more expensive and is therefore reserved for peak demand periods. All boilers maintain constant production costs regardless of the hour of operation. The optimization strategy prioritizes the cheapest gas boiler first, adding output from more expensive units only when needed to meet the total heat demand. One of the gas boilers must also undergo a scheduled maintenance period of between 30 and 60 hours, during which it is taken offline and produces 0 MW.

Scenarios

Scenario 1 (Heat only)

Scenario 1 uses a single district heating network with three gas boilers (GB1, GB2, GB3) and one oil boiler (OB1). The gas boilers are cheaper, while the oil boiler has higher production costs and is mainly used in peak demand periods. All boilers have constant production costs that do not depend on the hour of operation. Strategy for scenario one is to run the cheapest gas boiler first and then add production from the more expensive boilers if needed to meet the heat demand. There should also be chosen a period of 30 to 60 hours when one of the gas boilers will be down due to maintenance.

Scenario 2 (Heat and electricity)

Scenario 2 operates on a network with two gas boilers (GB1, GB3), one gas motor (GM1), and one electric boiler (EB1). The gas motor produces electricity that can be sold on the market, while the electric boiler consumes electricity that must be purchased. When electricity prices are high, the gas motor is prioritized, as selling electricity generates additional income. When prices are low, the electric boiler becomes the preferred unit, as buying electricity is cheaper. Unit selection is determined each hour by calculating and comparing the net production cost for

each unit.

The project is based on data delivered by Danfoss, including:

- Graphical representation of the district heating network
- Configuration parameters for each production unit (image, produced heat, produced/consumed electricity, primary energy consumption, production costs, CO2 emissions)
- Heat demand and electricity prices for a 14-day winter period with 1-hour resolution
- Heat demand and electricity prices for a 14-day summer period with 1-hour resolution

If there is time left, project can be extended by usage of other visualization libraries, storing data in a database, enabling or disabling units directly from the UI, running alternative configurations in Scenario 2, and implementing APIs and an API client for reading electricity prices from Energinet.dk.

1.3 Functional Requirements

The following tables outline the formal functional requirements for the system, translated into standardized system-level behaviors. The requirements are divided into three tables: one for each of the two operational scenarios, and a third covering general app navigation and UI requirements.

Table 1.1: Functional Requirements: Scenario 1

ID	**Related US**	**Requirement Description**	**Priority**	**Notes**
FR-001	HEAT - 78/79	The system shall allow the user to input data.	High	SDM&RDM
FR-002	HEAT - 78/79	The system shall allow the user to retrieve input data.	High	SDM&RDM
FR-003	HEAT - 79	The system shall allow the user to output data.	High	Database integration
FR-004	HEAT - 79	The system shall allow the user to upload assets from a CSV file.	High	Asset Manager
FR-005	HEAT - 82	The system shall allow the user to add an asset.	High	Asset Manager
FR-006	HEAT - 82	The system shall allow the user to edit assets.	High	Boiler definitions

Continued on next page

Table 1.1: Functional Requirements: Scenario 1

ID	**Related US**	**Requirement Description**	**Priority**	**Notes**
FR-007	HEAT - 188	The system shall allow the user to restrict the maximum heat output of each unit as follows: GB1 to 3.0 MW, GB2 to 2.0 MW, GB3 to 4.0 MW, and OB1 to 6.0 MW.	Critical	Capacity constraints
FR-008	HEAT - 146	The system shall allow the user to schedule the cheapest boilers first to fulfill the district's heat demand.	Critical	Cost-based dispatch
FR-009	HEAT - 146	The system shall prioritize boilers in the following strict order based on production costs: GB1 (510 DKK), GB2 (540 DKK), GB3 (580 DKK), and OB1 (690 DKK).	Critical	Dispatch order
FR-010	HEAT - 188	The system shall allow the user to use the more expensive oil boiler (OB1) only to supplement heat during peak demand periods when the gas boilers cannot fulfill the total demand.	High	Peak demand handling
FR-011	HEAT - 188	The system shall allow the user to optimize units separately for both a winter period and a summer period.	High	Seasonal optimization
FR-012	HEAT - 146	The system shall remove a designated gas boiler from the available pool of units during its scheduled maintenance period, ensuring it produces 0 MW.	Medium	Maintenance logic
FR-013	HEAT - 146	The system shall validate that the user's selected maintenance duration is a minimum of 30 hours and a maximum of 60 hours.	Medium	Maintenance validation

Continued on next page

Table 1.1: Functional Requirements: Scenario 1

ID	Related US	Requirement Description	Priority	Notes
FR-014	HEAT - 147/148	The system shall display a chart or graph that visually compares the total heat demand against the individual heat output contributions of GB1, GB2, GB3, and OB1 over time.	High	Result graphing
FR-015	HEAT - 188	The system shall allow the user to choose an obligatory maintenance period for one unit in both the summer and winter periods.	Medium	Maintenance scheduling

Table 1.2: Functional Requirements: Scenario 2

ID	Related US	Requirement Description	Priority	Notes
FR-016	HEAT - 40/91/92	The system shall allow the user to retrieve hourly electricity prices data from the database.	High	SDM&RDM
FR-017	HEAT - 91/92	The system shall allow the user to retrieve hourly heat demand data from the database.	High	Market data retrieval
FR-018	HEAT - 79/277	The system shall allow the user to upload assets from a CSV file.	High	Unit registration
FR-019	HEAT - 146	The system shall allow the user to limit the maximum heat output of each unit: GB1 to 3.0 MW, GB3 to 4.0 MW, GM1 to 5.3 MW, and EB1 to 6.0 MW.	High	Heat capacity constraints
FR-020	HEAT - 146	The system shall allow the user to account for electricity production and consumption capacities, specifically noting that GM1 produces up to 3.9 MW of electricity, while EB1 consumes up to 6.0 MW (-6.0 MW).	High	Electricity constraints

Continued on next page

Table 1.2: Functional Requirements: Scenario 2

ID	**Related US**	**Requirement Description**	**Priority**	**Notes**
FR-021	HEAT - 188/203	The system shall allow the user to calculate the net production costs for the electricity-producing/consuming units based on fluctuating electricity prices.	High	Dynamic net cost calculation
FR-022	HEAT - 188	The system shall prioritize running the gas motor (GM1) during periods of high electricity prices.	Critical	High-price optimization logic
FR-023	HEAT - 188	The system shall prioritize running the electric boiler (EB1) during periods of low electricity prices.	Critical	Low-price optimization logic
FR-024	HEAT - 188	The system shall actively prevent the gas motor (GM1) from running during summer periods with negative electricity prices (to avoid paying for produced electricity).	Critical	OPT
FR-025	HEAT - 188	The system shall recognize the electric boiler as an extra income source during negative price hours.	High	Negative price handling
FR-026	HEAT - 146	The system shall completely remove either GB1 or GM1 from the optimization calculations during its scheduled maintenance period.	High	Maintenance logic
FR-027	HEAT - 146	The system shall allow the user to select an obligatory maintenance period for either GB1 or GM1, and enforce a duration between 30 and 60 hours.	Medium	Maintenance scheduling
FR-028	HEAT - 146	The system shall display a graph comparing the total heat demand against the individual heat output contributions of GB1, GB3, GM1, and EB1 over time.	Medium	Result graphing

Table 1.3: Functional Requirements: General App Navigation & UI.

ID	**Related US**	**Requirement Description**	**Priority**	**Notes**
FR-029	HEAT - 149	Upon launch, the system shall display a home screen allowing the user to explicitly select between Scenario 1 and Scenario 2.	High	Startup Page
FR-030	HEAT - 149	The system shall allow the user to easily navigate between scenarios.	High	Scenario navigation
FR-031	HEAT - 91/92/233	The system shall allow the user to use a database system to save and retrieve data.	High	Database storage
FR-032	HEAT - 84/85	The system shall allow the user to dynamically activate or deactivate individual heating units from being considered in the optimizer's calculations.	High	Dynamic unit toggling
FR-033	HEAT - 84	The system shall allow the user to run a custom scenario with configurations set by the user.	High	Custom scenario configuration
FR-034	HEAT - 147/188	The system shall provide results of different configured scenarios for result comparison.	High	Comparison metrics

1.4 Non-Functional Requirements

The following table details the non-functional requirements (NFRs), which define the system's quality attributes, performance standards, and operational constraints.

Table 1.4: Non-Functional Requirements detailing system quality attributes.

ID	**Requirement Description**	**Notes**
NFR-001	The system shall complete all optimization calculations within 3 seconds of the user clicking 'Optimize'.	Optimization speed
NFR-002	The system shall completely render the resulting graphs within 3 seconds of the user clicking 'Optimize'.	Processing speed
NFR-003	The system shall efficiently load, process, and aggregate a full month of hourly data (approximately 720-744 data points) without UI freezing or lag.	Data volume handling
NFR-004	The system shall be fully cross-platform compatible, executing reliably on Windows, macOS, and Linux operating systems.	Cross-platform compatibility
NFR-005	The system shall present the entire user interface, including all menus, labels, and error messages, in English.	UI Language
NFR-006	The system shall calculate and display all financial values, production costs, and revenues strictly using Danish Kroner (DKK).	Currency localization
NFR-007	The system shall generate a highly readable visualization for a month of data, formatting the daily boiler usage as a stacked chart (matching the total heat demand on the Y-axis, and days of the month on the X-axis) alongside a daily cost graph.	Visualization clarity
NFR-008	The system shall not crash if user loses their internet connection.	System stability

Continued on next page

Table 1.4: Non-Functional Requirements detailing system quality attributes.

ID	Requirement Description	Notes
NFR-009	The system shall handle API failures by displaying a clear, understandable error feedback message or popup to the user.	Exception handling

1.5 System Requirements

The system is split into three parts: Front End, Back End and the Database. Both the Backend and the Database are run in a Linux Docker container. The Frontend is run locally on the user's machine. For a machine to run the project they need the following:

Software Specifications

Entire project: .NET 9.0

Backend: Docker Desktop (latest version)

Database: Docker Desktop with MySql Image (shall be installed automatically if missing)

Frontend: Required NuGet packages (all must be compatible with .NET 9.0):

- Avalonia.UI

Chapter 2

Scrum Implementation

2.1 Scrum Implementation

This chapter presents how the team implemented Scrum and its ceremonies from the very first week of development. Development consisted of six two-week iterative sprints. The team started with sprint planning, where they created a backlog of tasks and assigned story points to them. During each sprint, team members worked on their assigned tasks and held regular standup meetings to discuss progress and any obstacles they encountered. At the end of each sprint, the team held a sprint review to demonstrate their work and gather feedback, followed by a sprint retrospective to reflect on the process and identify areas for improvement. This iterative approach allowed the team to continuously improve its workflow and deliver a high-quality product by the end of the project.

The Scrum master was elected at the beginning, and because the member representing this role was highly skilled and the rest of the team agreed, he remained in this role until the end of the project. All team members agreed to use Jira. Jira was chosen because it allowed the team to use all features, including task management and proper story point assignment, without requiring additional subscriptions. Team members had extensive experience with Jira, as it allowed them to understand the complexity of the project more easily. The backlog was used to store future tasks, and certain tasks were transferred from the backlog to the sprint backlog during sprint planning. The board for each sprint consisted of several columns: “TO DO”, “IN PROGRESS”, “UNIT TESTING”, “UNDER REVIEW”, and “DONE”. Furthermore, one member integrated Jira with the IDEs, so when a team member started working on a new branch, the task automatically moved to “IN PROGRESS”. A crucial aspect of team cooperation was the approach to code reviews. Every task required a minimum of two reviewers. If a reviewer identified any issues or deficiencies, a change request was created in GitHub, and the branch owner had to resolve it before the reviewer would grant approval again.

The project was completed over six sprints. Each sprint consisted of the necessary Scrum ceremonies. Based on the project requirements, the team scheduled two standup meetings per week, specifically on Mondays and Wednesdays. These standup meetings remained consistent throughout the semester, serving as a short and structured way to synchronize progress, address problems, and ensure effective communication until the end of the project.

Sprint planning was held every other week before the start of a new sprint. Each planning

session was scheduled for four hours. For the first two sprints, planning took place on Mondays. After this period, the team decided to move sprint planning to Friday of the preceding week. Monday was a particularly busy day for all team members, and moving planning to Friday allowed the team to use the weekend for preparation, which made the transition between sprints smoother. With each subsequent sprint planning, less time was required as team members got better at planning, focusing, and deciding which tasks needed to be done and who was the right person for the job.

At the end of each sprint, the team held two final ceremonies: the sprint review and the sprint retrospective. Both ceremonies were held on Fridays.

Initially, all deadlines for tasks and pull-request reviews were set for Thursday evening. However, the team found this approach ineffective, and there was not enough time for reviews to be completed before the end of the sprint. The deadline for tasks was later moved to Wednesday evening, and the deadline for reviews to Thursday evening. This adjustment ensured there was enough time for all ceremonies held on Friday.

During the sprint review, team members presented what each of them had accomplished over the past two weeks. This session, along with the retrospective, was especially important for the team. It provided valuable insights into each member's progress, task preferences, and skill levels, allowing everyone to understand how the work was executed. The sprint retrospective was conducted through Jira. For each sprint, one team member established a table with three columns: "Start doing", "Stop doing", and "Keep doing". The team was given ten minutes to submit anonymous feedback, after which everyone reviewed and discussed the compiled topics. This ceremony turned out to be just as helpful as the sprint review. The retrospective was the ceremony that contributed the most to the team's growth and ability to cooperate effectively.

Chapter 3

Sprints

3.1 Sprint 1

Goal: First sprint focusing on the beginning of the project, including report writing and initial coding tasks.

Short summary: Both report and coding tasks were completed successfully. 2 bugs were not fixed. On report entity framework still persisted. During this sprint the team realized the importance of correct story point assignment and the need for better communication between team members. The retrospective revealed that the team needs to improve on time management and task estimation for future sprints (Sprint 1 planning, backlog, and retrospective can be found in Appendix D).

3.2 Sprint 2

Goal: Second sprint focusing on connecting the frontend and the backend, unit testing and implementation of core UI elements.

Short summary: This sprint there were no report tasks. On the other hand the team made good progress on the code side, getting closer to the first feasable demo. All tasks were successfully finished. Unit tests were written retroactively to Asset/Source services. The frontend was connected to the backend and Results/ResultLists were created. This sprint the team started experimenting with user stories that could not be finished in one sprint, because of significant complexity. In cases like this subtasks were assigned as if they were normal tasks. These large items were left in TODO for each sprint until all subtasks were done. Team members started figuring out the optimizing algorithm used by the system, and created the Asset Manager Window (Sprint 2 planning, backlog, and retrospective can be found in Appendix D).

3.3 Sprint 3

Goal: Third sprint focusing on the completion of the first demo, including the implementation of the optimization algorithm and the creation of a user interface.

Short summary: The team managed to complete all required tasks for the demo, even though one task had to be left for the next sprint, as it was not necessary for the demo and it took

up precious time from working on the demo. Minor tasks had to be assigned in the middle of the sprint. For example CSV creation for the demo was not planned, but it was necessary to show the results of optimization (Sprint 3 planning, backlog, and retrospective can be found in Appendix D).

3.4 Sprint 4

Goal: Fourth sprint focusing on LaTex report writing and implementing feedback received from the midterm evaluation.

Short summary: The team managed to implement most of the feedback received on the midterm evaluation and the project is wrapping up nicely. Members are going to focus on implementing extra requirements. Report got migrated from Word over to LaTeX so the team focused more on fitting it to the given template (Sprint 4 planning, backlog, and retrospective can be found in Appendix D).

3.5 Sprint 5

Goal: Fifth sprint focusing on report tasks as well as on small quality of life UI changes.

Short summary: The first week of this sprint was unfortunate as not all team members were in Denmark and could not fully concentrate. Despite the short time, the team managed to implement all tasks preparing for final sprint. Except for merge conflicts slipping through and making the main branch not working, there were not any big problems (Sprint 5 backlog and retrospective can be found in Appendix D).

3.6 Sprint 6

Goal: Sixth sprint focusing on report and code finalization.

Short summary: The team managed to finish all tasks for the final sprint, including report finalization and code cleanup. Overall, the project was a success and the team is proud of what they have accomplished (Sprint 6 planning and retrospective can be found in Appendix D).

Chapter 4

Solution (Technical Aspect)

This chapter outlines the steps undertaken to achieve the final solution, starting from the project description, describing the technical architecture in the process. The development process began with going through the provided Project Description, and rewriting it into a shorter version, keeping only the most important parts. The compact version was suitable for conducting the verb/noun analysis where class and method candidates emerged. The class candidates are able to be transformed into CRC cards outlining the responsibilities and collaborators of each class. The CRC cards are used to provide UML diagrams(Class,Package,Activity and Sequence).

4.1 Verb-noun analysis

Asset Manager (AM) should store its data in local files and supply other modules with it. Source Data Manager (SDM) should store dynamic data in csv files. Result Data Manager (RDM) should save result data in local CSV files. Optimizer (OPT) should calculate the best economical schedule of the heat production for a given period. It also secures heat availability to all buildings at any time. There are different units, each has a specific production cost, expressed in DKK per MWh produced heat. For heat only units there are no further expenses nor income. Electricity producing units should sell the produced electricity and have thereby an extra income depending on the electricity prices which can change from hour to hour. Electricity consuming units should buy the electricity and have thereby an extra expense, again depending on the electricity prices. Net production cost should be calculated differently for each unit type and compared to build priorities. The Data Visualization should display a graphical presentation of important source and result data.

Candidates for classes:

- Asset Manager
- Data
- Files
- Modules
- Source Data Manager
- Result Data Manager
- Optimizer
- Schedule
- Production
- Period
- Heat availability
- Buildings
- Time
- Electricity
- Units (GB1-EB1)

- Income
- Expenses
- Prices
- Net production cost
- Unit type
- Data Visualization
- Presentation
- Priorities

Candidates for methods:

- Store
- Supply
- Save
- Calculate
- Secures
- Has
- Sell
- Have
- Depending
- Change
- Buy
- Compare
- Display

4.2 Classes and Methods Mapping

Class Name	Associated Methods
AssetManager	store, supply
SourceDataManager	store (dynamic data), read CSV
ResultDataManager	save (results)
Optimizer	calculate (schedule), secure (heat availability), compare
Unit	sell (electricity), buy (electricity), calculate net production cost
DataVisualization	display (graphical data)

CRC Cards

Class: Asset Manager	
Responsibilities	**Collaborators**
• Store heater data	• Optimizer • Data Visualization • Unit

Class: Optimizer	
Responsibilities	**Collaborators**
• Calculates heating schedule	• Asset Manager • Source Data Manager • Result Data Manager

Class: Source Data Manager	
Responsibilities	**Collaborators**
• Stores market data	• Optimizer • Data Visualization

Class: Result Data Manager	
Responsibilities	**Collaborators**
• Save output data	• Data Visualization • Optimizer

Class: Data Visualization	
Responsibilities	**Collaborators**
• Display important data to the user	• Source Data Manager • Result Data Manager • Asset Manager

Class: Units	
Responsibilities	**Collaborators**
• Contains data for different heaters • Calculate net production costs	• Optimizer • Asset Manager

4.3 System Design

The system design, implementing the five modules(OPT,DV,AM,SDM and RDM), focuses on distinguishing data, services and the presentation layer. This was achieved by storing data in a Database, having a Backend API for services and a Frontend UI for the presentation layer. The layering helps create a modular solution, that is able to implement every module in a logical way, without affecting one another.

4.3.1 UML Diagram - Class Diagram

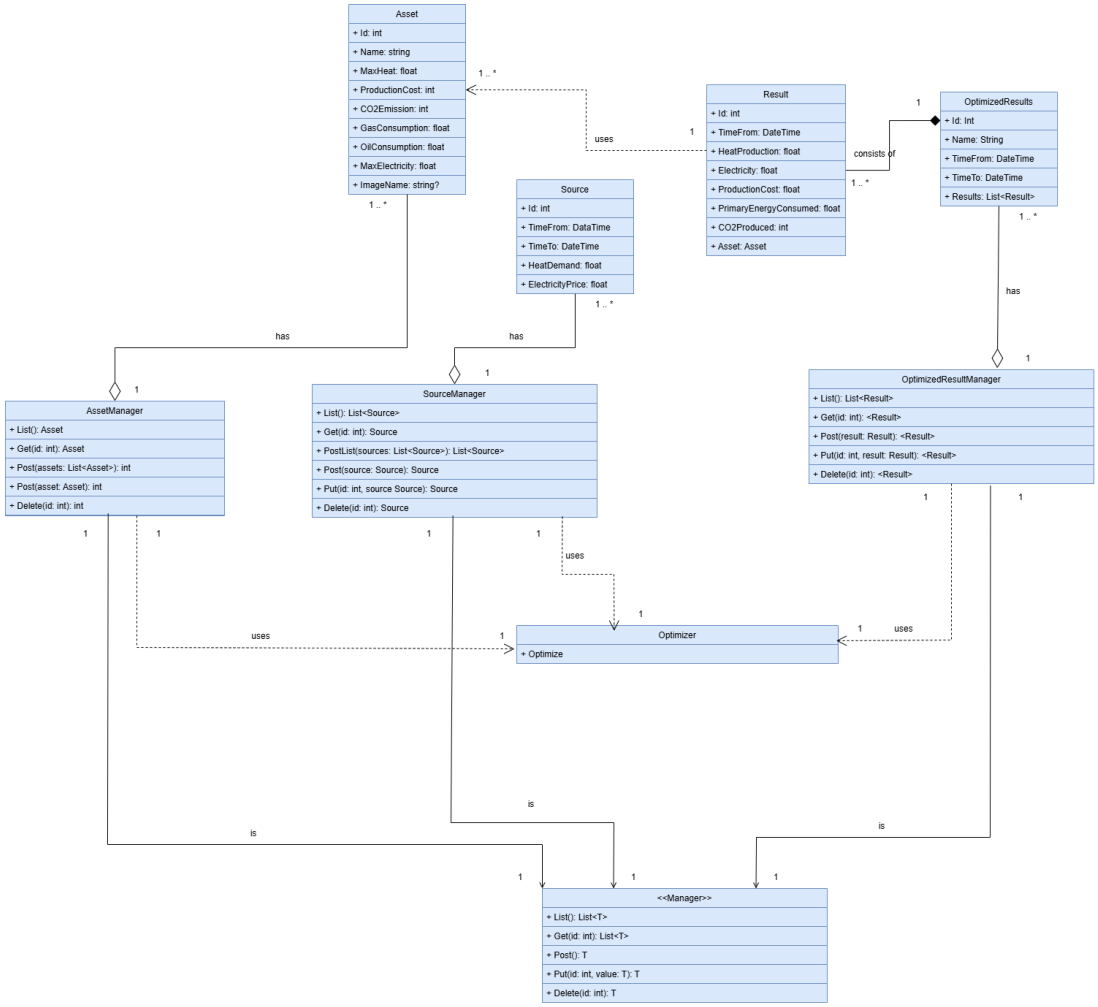


Figure 4.1: HeatItOn Abstract Class UML Diagram

The diagram (Figure 4.1) represents the unified system architecture. This is an abstract version that simplifies the structure down to core components, on which the system is based on. The simplicity of the UML helps with understanding the overall structure of the system, while showing all major classes.

4.3.2 UML Diagram - Package Diagram

The package diagram shown (Figure A.1) simplified the system into five packages.

Figure 4.2: Package UML Diagram - Five main packages of the system.

The first one is the Frontend package, containing the Asset, Result and Source Managers. This is the UI of the system, with what the user will interact. The Frontend is following the Model-View-ViewModel pattern, layering the different parts of each manager. This package connects with the next package through clients.

The second package, the Backend Controllers, contain the CRUD(Create, Read, Update, Delete) operations of each module. Through HTTP requests these can be accessed and will call the services in the next package.

Backend Services are the third package containing all the business logic (like the Optimization algorithm).

The forth package is the connection with the Database. It is stored in the Data layer of the Backend. The role of the package is to allow communication between the services and the Database.

The fifth and final package consists of the DB Models(Asset, Source, Result) that are stored in the Database.

This diagram shows a very simplified relationship with the most important parts of the overall system, giving an understanding of the overall workings and layers. Although it is very simple, it is a good starting point for understanding the entire architecture.

The five packages are the backbone of the implementation. Allthough they are much more detailed, this simplified version helps understanding the workings of the solution.

4.3.3 UML Diagram - Activity Diagram (Add Asset)

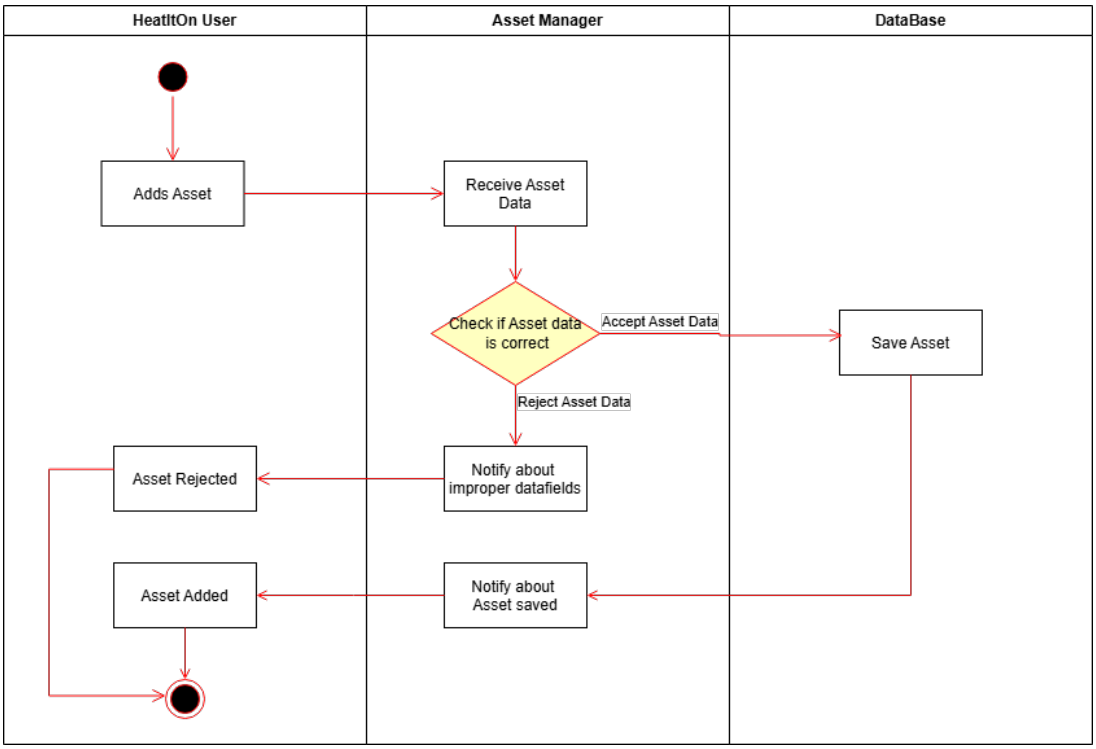


Figure 4.3: Add Asset Activity UML Diagram - Three parts and their actions for adding a new Asset.

The activity UML diagram (Figure 4.3) depicts the addition of a new Asset by the user into the system.

The activity is initiated by the user when they want to add a new Asset interacting with the Asset Manager. The filled out data is then verified by the Asset Manager. If the data was filled out correctly (like having a proper name or number fields having numbers) the Asset Manager will create the Asset and send it to the Database, who will save it. If the data was not filled out correctly, it will reject the received data. The activity ends with the user being notified of both cases, ending it. For a successful addition the Asset Manager will also display the new Asset instantly.

Adding an Asset is started in the Frontend. The Frontend clients will call the Backend that will handle the communication with the Database where the Asset will be saved. The Frontend deals with user input and handling the data as well.

This diagram helps getting an idea about how the communication, for adding Assets, works. This activity is crucial to the solution, as every Asset needs to be stored in the Database for the optimization to work.

4.3.4 UML Diagram - Activity Diagram (Import Sources)

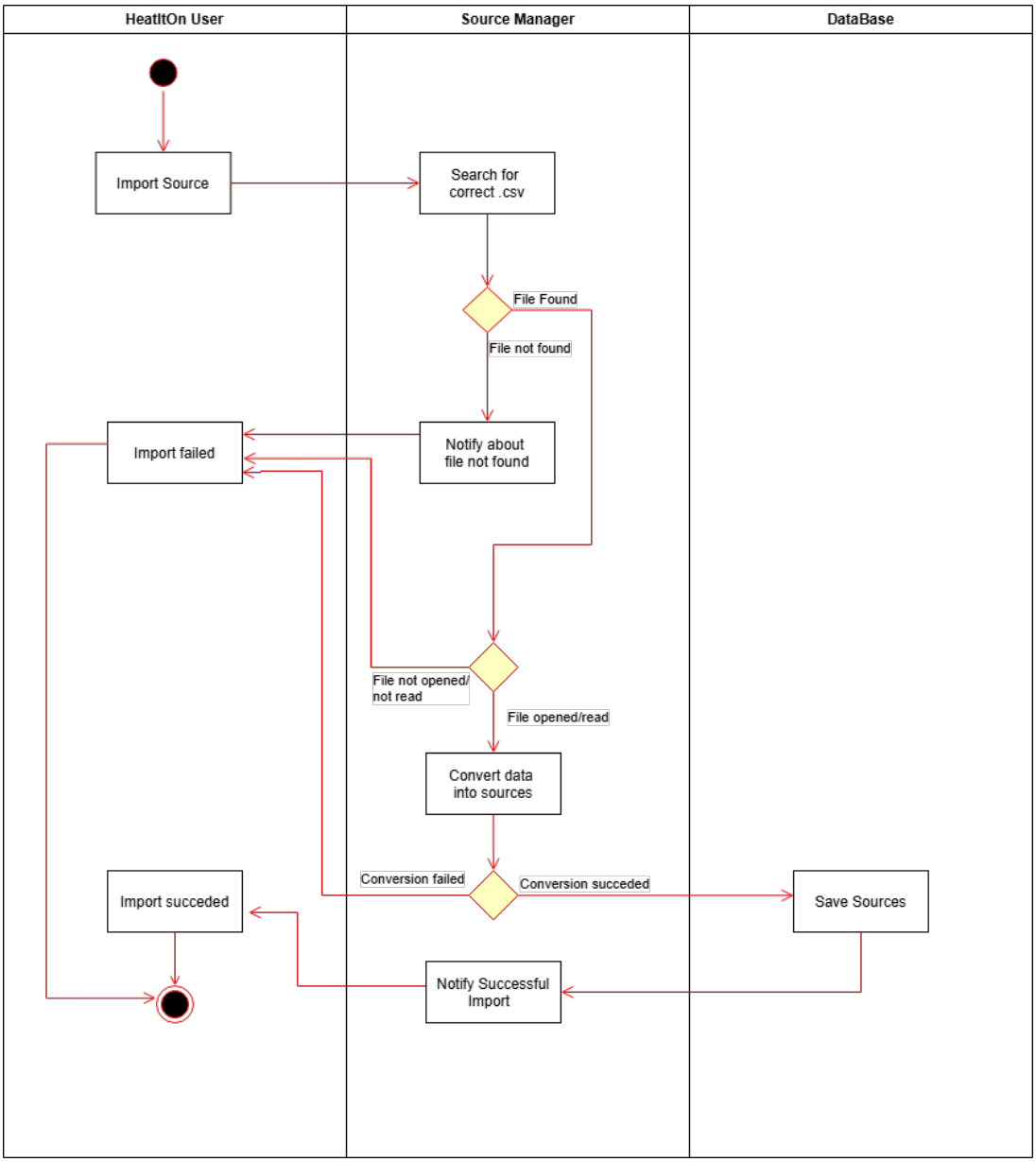


Figure 4.4: Import Sources Activity UML Diagram - Three parts and their actions for importing Sources from a csv file.

The activity UML diagram (Figure 4.4) depicts the importing of one or multiple Sources stored in a Comma Separated Value(csv) file.

Importing is initiated by the user interacting with the Source Manager UI. The Source Manager then will try to find the requested csv file. If the file is found the Source Manager will try to open and read the data in the Frontend. After a successful opening and reading the data will be transformed into proper Source models and will be sent over through the Backend to the Database for storage, notifying the user. If the file is not found or if the file cannot be opened or read properly, like if the csv is not structured properly, the importing process fails and the user is notified. The activity is concluded by notifying the user of either a successful or failed import,

the former is visible by a new data set appearing in the Source Manager.

This helps getting an understanding of how importing works for sources (but not only), and the workings of the Source Manager.

The Source Manager and Sources are the other crucial part of the system, without which, the optimization process could not be run.

4.3.5 UML Diagram - Activity Diagram (Delete Asset)

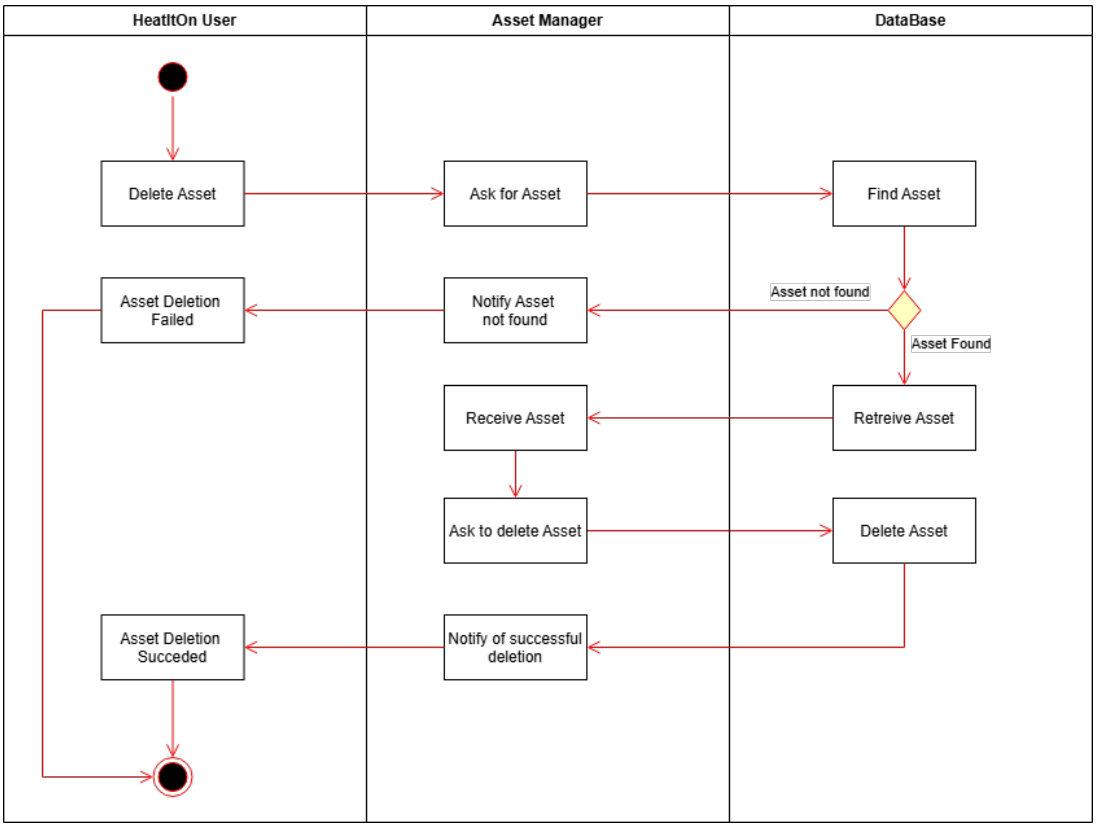


Figure 4.5: Delete Asset Activity UML Diagram - Three parts and their actions for deleting Assets.

The activity UML diagram (Figure 4.5) depicts the deletion of an Asset through the Asset Manager.

Deletion is initiated by the user through interacting with the Asset Manager UI. The Asset Manager will try to retrieve the Asset from the Database(through the Backend). If the Database finds the Asset it will return it to the Asset Manager. If the Database could not find the requested Asset, it will prompt the Asset Manager to notify the user ending the activity. If the Asset is found it will be retrieved to the Asset Manager who will request its deletion from the Database. The Database deletes the Asset and the Manager will notify the user of the activity being successful, after which it ends.

This activity diagram portrays another CRUD operation, in this case Deletion, in the system.

This helps getting an understanding of how deletion of objects works in the system, using Assets as an example. Deletion is crucial for the proper workings of the system.

Deletion is used with Assets, as seen on the diagram, but also with Results, where the user has the option to delete either of them.

4.3.6 UML Diagram - Activity Diagram (Optimize)

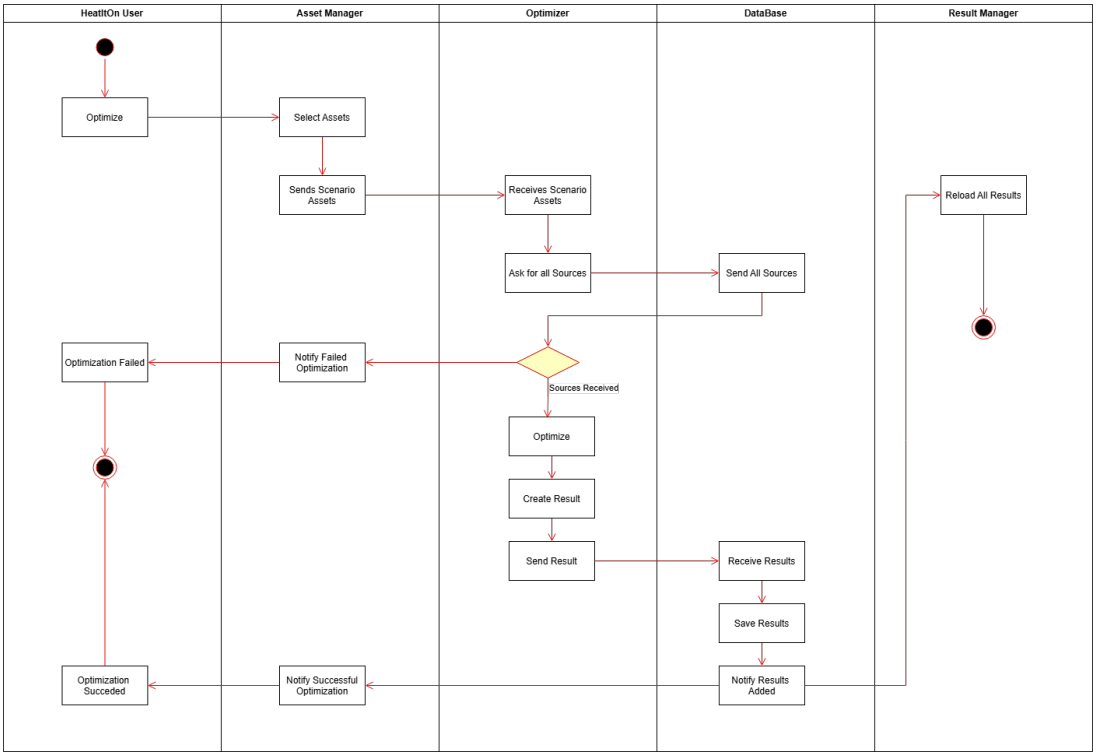


Figure 4.6: Optimization Activity UML Diagram - Three parts and their actions for deleting Assets.

The activity UML diagram (Figure 4.6) depicts the optimization process.

First, the optimization starts with the user, using the Asset Manager UI, selecting Assets and starting the optimization process. The Asset Manager will send the selected Assets to the Optimization module, which after receiving them, will call from the Database all stored Sources up to that moment. If no Source is received it will prompt the Asset Manager to notify the user and end the activity. The Optimizer has no UI elements and is completely in the Backend. If it receives Sources, it will run the optimization algorithm, which will create a Result. The new Result is sent and stored in the Database that will notify both the Asset Manager(that in turn notifies the user) and the Result Manager that there is a new Result ready to be displayed, ending the activity.

This activity diagram proves its relevancy by demonstrating the optimization activity, which is the entire purpose of the system. While it is not enough for a full understanding of the optimization process, the diagram illustrates the different overall steps happening.

Optimization is the part that interacts with every module of the system. It uses directly or indirectly every implemented Manager, and all the activity diagrams(and all their depicted actions) are just helping the optimization process depicted here.

4.3.7 UML Diagram - Sequence Diagram (Optimize)

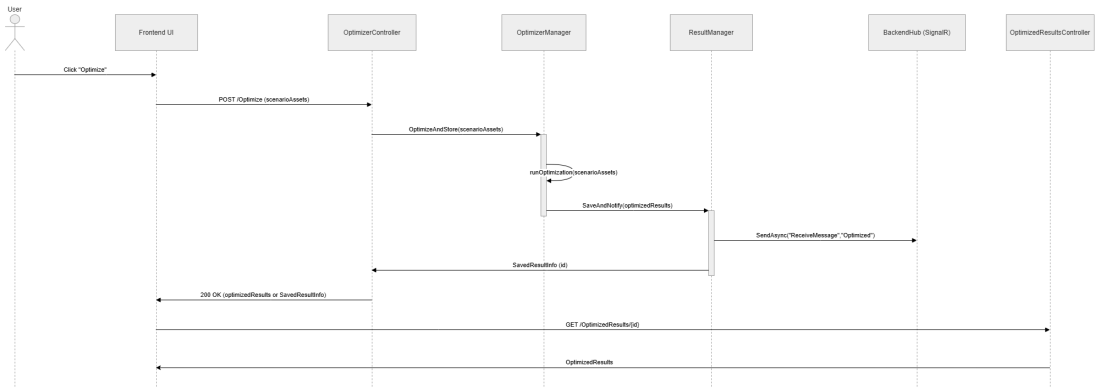


Figure 4.7: Sequence UML diagram of optimization process

The sequence UML diagram (Figure 4.7) depicts the optimization process. The participants are: User, Frontend UI, ‘OptimizerController‘, ‘OptimizerManager‘ (runs the algorithm), ‘ResultManager‘ (saves results and sends notifications), ‘BackendHub‘ (SignalR) and ‘OptimizedResultsController‘ (for later retrieval). The flow is: the user clicks Optimize in the frontend, the frontend sends the selected assets to ‘OptimizerController‘ with ‘POST /Optimize‘, ‘OptimizerController‘ calls ‘OptimizerManager‘ to run the optimization, ‘OptimizerManager‘ produces an ‘OptimizedResults‘ object and gives it to ‘ResultManager‘ to save, ‘ResultManager‘ writes the result to the database and asynchronously notifies clients via ‘BackendHub.SendAsync("ReceiveMessage","Optimized")‘, the controller returns either the full result or a small confirmation (‘SavedResultInfo‘ with the new id) to the frontend, the frontend can then request the stored result with ‘GET /OptimizedResults/id‘ from ‘OptimizedResultsController‘ to display the full persisted data. If the optimizer cannot find required source data, it returns an error/notification back to the frontend and the activity ends with the user being informed. In the diagram the ‘OptimizerManager‘ and ‘ResultManager‘ are shown as manager boxes that group the backend logic (they represent the real services and saving/notification steps) so the sequence stays easy to read while matching the actual system behavior.

This sequence diagram shows the main flow: how a user action becomes an optimized schedule and is delivered to the UI, connecting the user, backend computation, storage and real-time updates so the reader sees the end-to-end behavior without needing the algorithm details.

How this maps to the code:

- The optimizer endpoint is in OptimizerController, the OptimizerController receives the POST /Optimize request.
- The optimization work runs in OptimizerService (this is the diagram’s OptimizerManager).

- Saving the results is done by OptimizedResultsService (the diagram's ResultManager).
- Real-time notifications go through SignalR in BackendHub, controllers use IHubContext<BackendHub>.SendAsync to send messages.
- Fetching stored results is handled by OptimizedResultsController.
- The database model and migrations are in BackendDbContext.
- Centralized error handling (turning exceptions into HTTP responses) is implemented in ExceptionHandlingMiddleware.

4.3.8 UML Diagram - Sequence Diagram (Refresh)

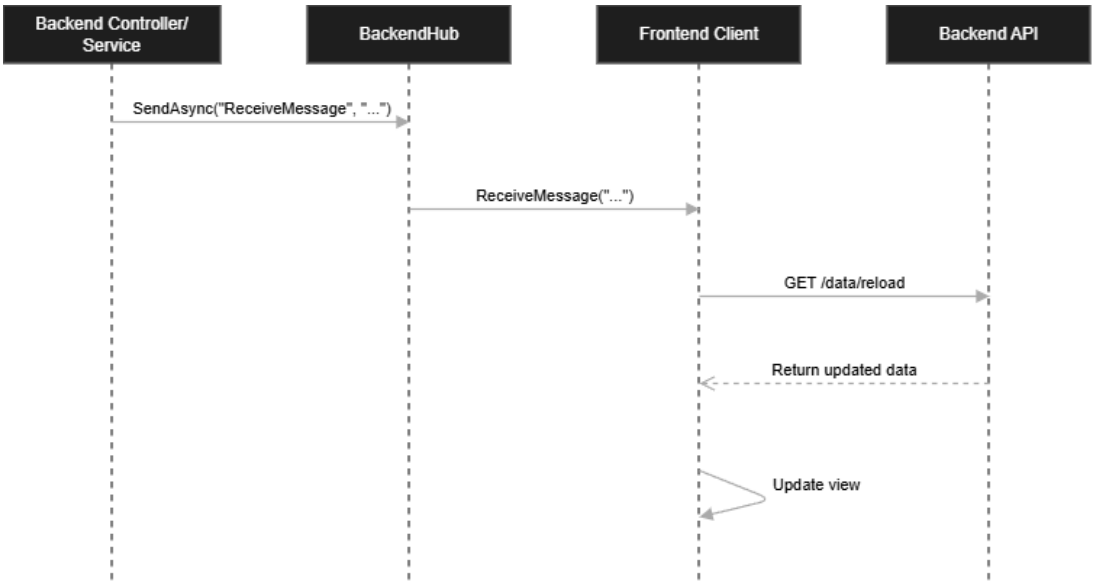


Figure 4.8: Sequence UML diagram of refreshing process

The refresh sequence diagram is relevant because it shows how the frontend stays synchronized with changes made in the backend, which is an important part of the system's behavior. It demonstrates the full update flow after a backend action has changed some data: the backend first sends a SignalR notification and then the frontend reacts by loading the latest data again. This makes the diagram useful because it explains how the user sees updated information without manually reloading the page and it also shows the difference between pushing a notification and pulling the new data. The diagram connects directly to the implemented solution in the codebase: controllers and services use 'IHubContext<BackendHub>' to call 'SendAsync(...)', the frontend receives the 'ReceiveMessage' event and then it requests the updated data from the backend through endpoints such as 'OptimizedResultsController' or another reload request. In this way, the diagram reflects the real implementation and shows how real-time updates are handled in the system.

4.4 System Technical Architecture

For the system architecture, from a technical standpoint, C# [5] was chosen as the base technology stack. This was chosen as its .NET 9.0 version is suitable for the development of this system with its simplicity, the familiarity of the team and its object-oriented imperative style. Every other piece of technology is related to .NET and was chosen based on their respective compatibility with the language.

Technology Stack

The first chosen piece was the Database that is running MySQL [2]. The role of the Database is to store the system data independently from other parts of the system(Backend and Frontend). It stores relevant user data, using models, like the Assets, Sources, Results of each generator and Results of optimization runs. The Database communicates with the Backend using MicrosoftEntityFramework. The Frontend cannot communicate directly with the Database, it must do it through the Backend. The Database allows data storage that is independent from runs. The system can work with multiple users at the same time or with one user in different runs, storing all the data in the Database, where it will remain until deletion.

The second chosen piece of technology stack is for the Frontend UI. For this AvaloniaUI was chosen for its good compatibility with .NET and modularity. Besides this, the Frontend is following the Model-View-ViewModel (MVVM) design pattern.

Layering

The system is divided into three main parts, Frontend, Backend and the Database. Each serves a different role and, besides the Database, they are further separated for proper readability and modularity.

The Frontend, as stated earlier, follows the MVVM pattern. The Models, together with the Data (containing file I/O and clients) form the overall data and business layer. The View forms the presentation layer, containing all the UI elements. The ViewModel creates a connection layer between the two other parts.

The Backend is following a .NET web application template. Its role is to create a connection between the Frontend and Database with an Application Programming Interface (API), and to handle overall business logic for the entire system. For modularity, this is achieved through serious layering. The Backend consists of six main parts. Controllers handle incoming/outgoing traffic to/from the Database and Frontend and will call the appropriate Services. Services contain all the business logic of the system(like optimization or add Asset). Both Controllers and Services inherit from two Interfaces, as they share similarities (like CRUD operations). The Data layer is composed of two other parts, Models(storing data models) and Data(storing DB Migrations and DB Context which is the main connection point to the DB). The presentation layer is completed by the last element, Hubs, which ensures real time communication with the Frontend application (instantly notifies to refresh the Frontend). Although the Backend is divided into many pieces,

this was necessary for modularity, readability and because of complexity as well. This way each small section is responsible of one part, making replacement easier. In this solution, if one wants to hypothetically change the entire optimization algorithm, they only need to rework the Optimization Service, without having to touch any other parts.

Architectural Style

The implementation follows multiple coding and design principles. The main set of principles that the project adheres to, is SOLID. This happens in both Frontend and Backend and is visible in many parts, like interfaces or being able to add new features without having to change code. SOLID principles help create an Object-Oriented design. But besides these, the project follows DRY (Do not Repeat Yourself), KISS (Keep It Simple, Stupid), YAGNI (You Aren't Gonna Need It) and SoC (Separation of Concerns).

The technology stack, the layering and the architectural style used are all necessary for the proper running of the system. While the technology stack was kept simple by only using what was necessary for achieving requirements, the layering and style help build a good quality project that is easy to understand, change and complete. But extensive layering also helps following the style, by breaking up the large task into small sections creating a *Divide et impera* style of development, making it easier to adhere to the principles.

Chapter 5

Implementation and Technical Realization

HeatItOn is divided into three sections, and two versions. First there is the midterm evaluation demo version, and then there is the final version. We will go through both to see the first implementation and the improvements and changes that we made for the final version.

5.1 Frontend Implementation

Our Frontend Implementation is desgined with the plant operators in mind, keeping it simple and user friendly. To achieve this, we started with a UI-sketch where we tried to come up with an easy to use and easy to understand concept, that we tried to achieve in our implementation.

The application is split into three different tabs. The first one, that the user sees when they launch the application is the Assets tab. Here the user can manage and see all the existing assets [7], they can import or export assets as csv's or manually add/remove or edit specific assets one by one. This page also has three preset toggles(Scenario 1, Scenario 2 and Custom). As the Assets page is where the operator will start the optimization process, by selecting the asset combination they want to optimize with, the toggles merely select from the assets the ones that are in that scenario. If they wish to optimize for a new scenario, they can select the Custom toggle, allowing them to run the optimization process for any scenario. After selecting the desired assets the user has to click on the Optimize button that will start the optimization process for the selected assets.

The list of assets is connected through the backend with the database, each modification being done live in the database. The optimization process from the backend is called from the frontend by clicking a button. The Assets tab will send the Optimization algorithm the assets selected to be optimized every time the button is clicked.

The second tab, the Results tab is where, once the user started an optimization, the result of that run will appear. On the left side, they can see and select out of a list of all results. When they select one result, the user can see the hourly result in a table format or view the data in different graphs. They can also export them into csv's. If the user would like to focus on a specific time period, not the entire optimiziation run, they can use a date-time picker to select from when to until when they want to see the result data, which will affect the result table and

the graphs as well.

The tab loads all the optimization-results from the database using the backend services.

The third tab, Sources, is where the user will upload/export or even edit source data. On the left, you can see a list of uploaded csv's again. When the user selects one, they can also visualize the source data in a few simple charts or see the entire source data in a table. The cells of the table are modifiable so they can also edit the source data.

This tab uploads all sources directly to the database, using the backend services, while the csv file's are handled locally.

Our goal with each tab was acheaving an easily navigable and understandable UI, while giving all the necessary tools for the plant-operators/users for adequate management. Although we only received source data for four weeks and two asset scenarios, we tried to create a scalable, modular UI, that can handle dozens of assets and years worth of source data. As a side goal we also wanted to achieve a simple yet modern and visually appealing look.

5.1.1 Midterm Version

For the midterm demo, we prioritised usability over aspect. As this version was a mere proof of concept, and its goal was to plant a picture of our way of implementing it and to show what we think is possible, some aspects of the overall user experience was ignored so we can focus on actual functionality.

This means that the UI for this version has a very generic, template look. Buttons are rough and placed where we could fit them, while icons are uninteresting and unintuitive. A majorly different approach for displaying assets, results and sources was used in this version. Instead of the frontend lists updating to changes accordingly(like uploading csv's or generating results) in real time, we added a refresh button that, when clicked, reloads all the data from the database. This solution was a compromise made for this version, and eliminating it, a focuspoint for the final version.

The Assets tab was fully functional, and only needed some visual and aesthetic improvements (see Appendix D).

The Results tab in this version is defined by minimalistic graphs, that made it hard to visualize the results. Our three graphs(electricity use/production, heat production and CO2 production) were a good start but we realised we need a lot more for proper visualization. (see Appendix D).

The Sources tab functioned as expected but the possibility to edit the csv data was missing from this version.

Overall the demo was a very solid proof of concept that helped us learn what we were missing for the final version while also showing us the direction we are going.

5.1.2 Final Version

The final version focuses more on user experience with minor fuctionality improvementse when compared to the midterm implementation. The primary goal of this version was to improve usability, visual appearance, and introduce additional statistical aspects while maintaining a consistent and scalable structure.

A significant improvement is the change from a manual data refresh system to a dynamic, responsive interface. In contrast to the midterm version, where a dedicated refresh button was required to update data, the final implementation ensures that all lists and views update automatically in response to changes in the database. This enhancement improves workflow efficiency and eliminates unnecessary user interaction.

The visual design of the interface has been improved. The final version adopts a modern, structured design with consistent colors, clearer text formating, and improved spacing. Asset cards have been redesigned to display key information in a meaningful and readable format, including maximum heat, production cost, CO_2 emissions, and electricity generation/consumption. Additionally, asset categories such as gas, oil, electric, and motor are visually distinguished through icons and labels, allowing for faster recognition and comparison.

The scenario selection system has been refined and better integrated into the workflow. Instead of simple toggles, the final version introduces clearly defined scenario panels with descriptive labels such as “Peak Load,” “Eco Mode,” and “Custom Configure.” Each scenario communicates its purpose more effectively, enabling more intuitive decision-making. Furthermore, visual indicators highlight selected assets within each scenario, improving clarity during configuration.

Additional controls have been add, including a “Manager” and an “Optimize” button. The Manager section serves as a centralized interface for creating, importing, and exporting assets. These improvements lead to a clearer workflow, guiding the user from setup to execution in a more structured way.

The Results tab has undergone major enhancements in both data presentation and analytical depth. A summary dashboard has been introduced at the top of the page, presenting key performance indicators such as total heat production, net electricity balance, total CO_2 emissions, and total production cost. These aggregated metrics provide immediate insight into optimization outcomes without requiring detailed inspection of raw data.

The data display has also been improved with better formatting, clearer column labeling, and pagination support for large datasets. The addition of pagination ensures that even large datasets remain readable and manageable.

The charting system represents one of the most significant functional improvements. While the midterm version included only basic visualizations, the final version introduces a comprehensive set of charts:

- A stacked area chart showing heat production per asset over time, including a total heat overlay

- A production cost per hour chart, allowing detailed financial analysis
- A generator share pie chart, illustrating the contribution of each asset within a selected time range
- An electricity usage and generation chart, displaying hourly electricity balance in MWh

These visualizations provide a deeper understanding of system behavior and allow users to analyze performance trends, cost distribution, and resource utilization more effectively. Interactivity elements, such as tooltips and time-range filtering, further enhance data exploration. The date and time filtering functionality has been refined, with a more structured input system for selecting precise time intervals. Filtering affects both tabular and graphical outputs, ensuring consistency across all displayed data.

The Sources tab has also been expanded with important new functionality. Unlike the midterm version, the final implementation allows direct editing of source data within the table. This feature enables on-the-fly adjustments without requiring external file modification. The table design has been improved for clarity, and pagination has been introduced to handle large datasets efficiently. File management capabilities have been enhanced, including clearer separation between uploaded files and active data views. Import and export functions remain available but are now better integrated into the interface.

Overall, the final version achieves a significantly higher level of usability, clarity, and analytical capability. The transition from a proof-of-concept interface to a polished, production-oriented design is evident across all tabs. The system now provides a comprehensive toolset for asset management, optimization execution, and result analysis, while maintaining a scalable and user-centered design approach.

5.2 Backend Implementation

5.2.1 Midterm Version

The backend is implemented using ASP.NET Core 9.0 and follows a layered architectural design that emphasizes modularity, scalability, and separation of concerns. The structure consists of Controllers, Services, Interfaces, Models, and a Data layer, each responsible for a distinct part of the system functionality.

Architecture and Main Components can be found in Appendix D.

The backend is organized into clearly defined layers:

- **Controllers**: Provide RESTful API endpoints and act as the entry point for client requests. Controllers are responsible for handling HTTP requests, binding input parameters, and returning appropriate responses.
- **Services**: Implement the core business logic. Each service corresponds to a domain entity, such as assets, source data, and optimization results. Services coordinate data processing

and communicate with the persistence layer.

- **Models**: Represent domain entities and map directly to database tables. Examples include `Asset`, `Source`, `Result`, `ResultList`, and `OptimizedResults`.
- **Data Layer**: Contains the Entity Framework database context (`BackendDbContext`) and migration files. This layer manages communication with the MySQL database.

This architecture ensures that responsibilities are clearly separated, enabling easier maintenance and extensibility.

API and Controller Design

During the midterm the backend exposed basic REST endpoints (CRUD) implemented ad-hoc, these were formalized in the final version through generic controller/service interfaces (see the **Final Version** subsection).

All controller actions are implemented asynchronously using `Task<IActionResult>`, allowing efficient handling of concurrent requests. Controllers delegate processing tasks to the service layer, keeping the API layer thin and focused on HTTP concerns.

Business Logic Handling

All business logic is implemented within the service layer, which acts as the core processing unit of the backend. Each service implements the generic `IService<T>` interface, providing consistent operations across all entities.

Specialized services, such as the `OptimizerService`, encapsulate domain-specific logic related to heat production optimization. This includes processing input data, calculating results, and coordinating interactions between assets, source data, and result data.

The separation of business logic from controllers improves maintainability and enables independent testing of core functionality.

Data Processing and Validation

Data processing is primarily handled within the service layer. Incoming data from API requests is validated before being persisted or processed further. Validation mechanisms include:

- Strongly typed C# models for structured data representation
- Parameter binding and validation features provided by ASP.NET Core
- Logical validation rules implemented within service methods

This approach ensures that invalid or inconsistent data is prevented from entering the system and maintains overall data integrity.

Database Communication

The backend communicates with the MySQL database using Entity Framework. The `BackendDbContext` manages entity tracking, query execution, and persistence.

- Models are mapped directly to database tables
- LINQ is used for querying and manipulating data
- Asynchronous database operations improve performance and scalability

Services interact with the database context to perform all data operations, ensuring controlled access to the persistence layer and preventing direct database access from controllers.

System Behavior Support

The backend supports the required system behavior by coordinating interactions between different system modules. Controllers expose endpoints for managing assets, source data, and optimization results, while services execute the corresponding logic.

The inclusion of a dedicated optimization service enables the processing of time-series data for heat demand and electricity prices, supporting the generation of optimized production schedules.

Deployment Considerations

The backend is deployed using Docker, which provides a consistent and isolated runtime environment. Containerization ensures that dependencies, configurations, and runtime settings are standardized across development and production environments.

This approach improves portability, simplifies deployment, and enhances the reliability of the backend infrastructure.

Overall, the backend implementation follows best practices for modern web application development, emphasizing modularity, scalability, and maintainability. The layered architecture ensures that the system can effectively support the required functionality while remaining adaptable to future enhancements.

5.2.2 Final Version

The final backend version extends the original layered ASP.NET Core architecture with additional runtime functionality and tighter system integration. We introduced two supporting components: a centralized exception-handling middleware and interface-based service contracts that define abstraction boundaries between controllers and services.

These additions work together with the existing layers as follows: controllers accept HTTP requests and delegate work to services; services implement business logic and interact with the data layer (EF ‘BackendDbContext‘); both controllers and services rely on the ‘Interfaces‘ abstraction and dependency injection; the ‘ExceptionHandlingMiddleware‘ sits early in the request pipeline, catching exceptions from controllers and services, logging them, mapping domain errors to HTTP status codes, and returning compact JSON error responses. As a result, many local try/catch blocks were removed and method implementations became easily readable.

- **Exception handling**: Centralized in `ExceptionHandlingMiddleware`. It intercepts exceptions thrown by controllers and services, logs them, maps domain errors to appropriate HTTP status codes, and returns a compact JSON payload (for example `{"error": "<message>","stackTrace":"<stackTrace>"}`) with `Content-Type:application/json`. This provides consistent, machine-readable error responses for clients and prevents raw error details from leaking.
- **Interfaces**: Provide clear abstraction boundaries for controllers and services (e.g., `IController <TCreate,TUpdate>`, `IService<T>`). Using interfaces enables dependency injection, easier unit testing, and consistent error-typing so the middleware can map domain exceptions to appropriate HTTP responses.
  - The generic controller interface formalizes standard CRUD operations used across controllers:
    - `List()` for retrieving collections of entities
    - `Get(int id)` for retrieving a single entity
    - `Post(TCreate model)` for creating new entries
    - `Put(int id, TUpdate model)` for updating existing entries
    - `Delete(int id)` for removing entries
- **Controllers**: Handle request/response concerns and remain thin — delegating business rules to services and relying on interfaces for testability, validate inputs.
- **Services**: Contain core business logic, interact with the data layer, and throw well-defined domain exceptions when needed. They do not implement HTTP concerns or repeated try/catch logic anymore.
- **Data layer**: The Entity Framework `BackendDbContext` and related repositories/models remain responsible for persistence and are used by services through interface-driven abstractions.

5.3 Data Management

5.3.1 Midterm Version

Data management is implemented using a relational database approach supported by MySQL, combined with an object-relational mapping layer provided by Entity Framework.

Data Structure

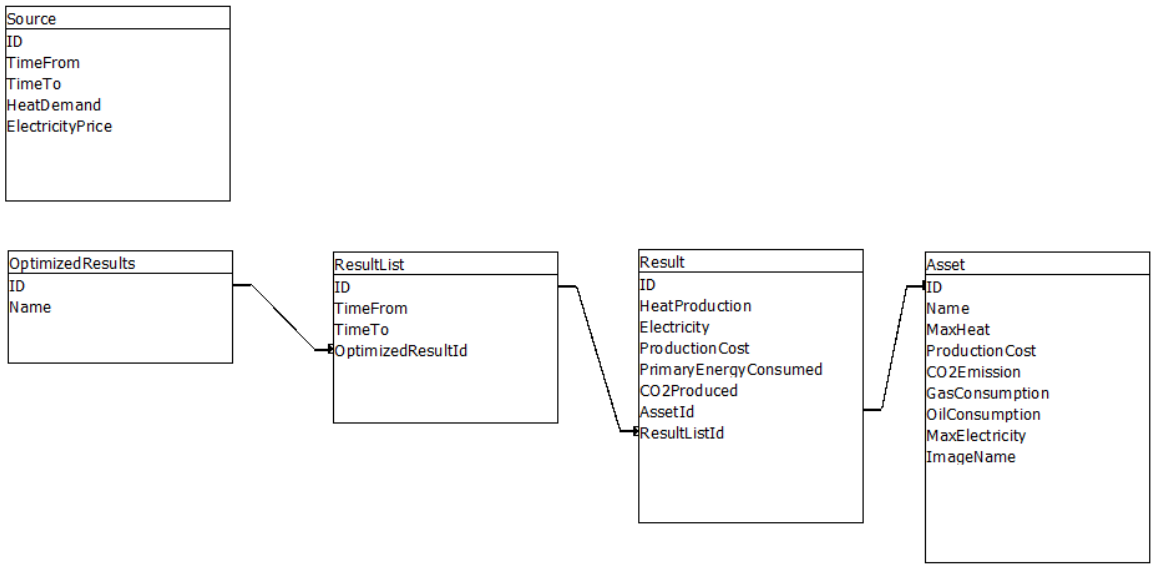


Figure 5.1: Database schema illustrating the relationships between Source, OptimizedResults, ResultList, Result, and Asset tables.

The data model is organized into five primary tables: Source, OptimizedResults, ResultList, Result, and Asset, each representing a distinct aspect of the system domain.

- The Source table stores time-based input data, including HeatDemand, and ElectricityPrice. This structure supports time-series data used for optimization processes.
- The OptimizedResults table represents a container for optimization runs during the selected time period.
- The ResultList table links individual time intervals to a specific optimization result through the foreign key OptimizedResultId. Each entry contains results for separate assets within an hourly time frame, with attributes such as TimeFrom and TimeTo defining the interval.
- The Result table contains detailed output data generated by the optimization process. Attributes include HeatProduction, Electricity, ProductionCost, PrimaryEnergyConsumed, and CO2Produced. This table also includes foreign keys AssetId and ResultListId, establishing relationships with both Asset and ResultList. It stores the specific performance metrics for one asset in one hour.
- The Asset table defines the characteristics of generators or boilers, including MaxHeat, ProductionCost, CO2Emission, GasConsumption, OilConsumption, MaxElectricity, and ImageName.

The relationships between these tables follow a hierarchical structure:

- One OptimizedResults entity relates to multiple ResultList entries.
- One ResultList entry relates to multiple Result records.
- Each Result references a single Asset.

Primary keys and foreign keys ensure referential integrity across all tables. For example, Result.AssetId must correspond to a valid entry in the Asset table, and ResultList.OptimizedResultId must reference an existing optimization result.

Integration with Backend

The data structure integrates directly with the backend through Entity Framework. Each database table is mapped to a corresponding C# model, enabling seamless interaction between the application logic and the database layer. Entity Framework provides abstraction over raw SQL queries, allowing developers to interact with the database using LINQ queries and strongly typed objects. The DbContext component manages database connections, entity tracking, and query execution, ensuring consistent data access patterns across the application. Migration tools within Entity Framework support schema evolution, allowing controlled updates to the database structure while maintaining data integrity.

Data Management Environment

The database is deployed using Docker, ensuring a consistent and reproducible environment. The use of Docker also facilitates easy deployment, testing, and version control of the data management infrastructure.

Overall, the data management approach combines a well-defined relational schema, validation mechanisms, and seamless backend integration to ensure reliability, scalability, and maintainability of the system.

5.3.2 Final Version

The recent database updates were minor and mainly aimed at improving data quality. We added and refined several model-level validation rules using data-annotation attributes (such as Required, Range, and MaxLength) to prevent invalid or inconsistent values from being saved. We also introduced a unique constraint for the Source entity in the DbContext configuration to avoid creating duplicate entries. These changes strengthen validation and consistency across the system without altering the overall database structure.

Chapter 6

Discussion and Reflection

This chapter discusses the software engineering practices and processes adopted throughout the project. It also contains potential future work that could further improve the product.

6.1 Scrum

"Scrum is a lightweight framework that helps people, teams and organizations generate value through adaptive solutions for complex problems."[6] Reflecting on the team's use use of Scrum, the framework had a positive impact on the team's workflow and collaboration. Compared to the previous semester project, where the work was less structured, Scrum introduced a level of organization that proved genuinely useful to the team.

Sprint Planning helped the team stay aligned by dividing tasks early and estimating effort through story points, which made it easier to identify when a sprint was becoming overloaded. Daily Scrums, held two times a week, improved our cooperation. Team members were more aware of each other's progress and blockers, which reduced situations where work was duplicated or misunderstood.

The Sprint Retrospective proved to be helpful for the team. Using the Start Doing, Stop Doing, and Keep Doing tables in Jira, the team was able to identify and discuss issues such as reviews being rushed, tasks lacking clear descriptions, and errors that made it past the review stage. Based on this specific changes were agreed on for the following sprint. This led to gradual but visible improvements in how the team cooperated over time.

Overall, Scrum contributed to better organisation, more open communication throughout the project.

6.2 Requirements Engineering

"Requirements engineering is the process of understanding and defining what services are required from the system and identifying the constraints on the system's operation and development."[6] Practises used during this project:

- User stories - Helped the team understand the user perspective and ensure the application was developed with real user needs in mind, contributing to a more user-friendly product
- Product Backlog - Provided a full and organised list of all tasks in one place, making it easier to manage and prioritise work across the project

- Functional requirements - Ensured all members of team understood what the software is supposed to do
- Non-functional requirements - Ensured all members of team understood the quality standard the software needed to meet
- Story points - Estimated the effort for all the tasks
- Sprint backlog - Helped the team stay focused on tasks chosen from the backlog for the current sprint
- Reviews with the team - Allowed each member to validate certain tasks helping to reveal and prevent mistakes and errors
- Definition of Done - Determined what finished task mean (code, testing, reviews)

In the future, all of these practices will be used again. User stories will be written in more detail, the product backlog will be prepared more thoroughly at the beginning of the project, and reviews will be taken more seriously.

6.3 Refactoring

Throughout the project, several refactoring improvements were made to keep the codebase clean and maintainable. CSV-related functionality was combined into a single unified class to reduce duplication. A centralized error handler was introduced, removing repetitive try-catch blocks from services and controllers. Data annotations in the models were expanded for better input validation, and interfaces were added to improve code structure. Unused imports were removed, formatting was cleaned up, and inheritance structures were reviewed and adjusted where needed.

6.4 Code review

During this project, two reviewers were assigned to every task in Jira and GitHub. Both had to either approve or request changes, and all issues had to be resolved before merging the work into the main branch. This practice helped improve overall code quality and catch bugs early in the process.

However, this practice came with some challenges. Early in the project, some reviews were delayed or not done properly, which occasionally led to mistakes slipping through. To address this, the team introduced a deadline system midway through the project. All tasks had to be completed by Wednesday of the second sprint week, and all reviews had to be finished by the following Thursday. This was not only to prevent reviews from being rushed at the last minute, but also to ensure Fridays could be dedicated fully to the Sprint Review and Retrospective, rather than still catching up on unfinished work.

6.5 Testing

During the project, unit tests were written for every task that involved business logic. Testing was part of the Definition of Done, meaning tasks could not go for review until the necessary tests were written.

Manual end-to-end testing was conducted to ensure the application ran and behaved as expected. For tasks involving the database, Postman was used to test every endpoint after new code was added or modified.

Unit tests were written for the backend modules covering a range of scenarios. For example, positive cases confirmed that data could be added and retrieved correctly, negative cases verified that incorrect values were not returned, and edge cases covered situations such as attempting to operate on non-existent entries or passing invalid input.

Unit testing proved to be one of the more challenging parts of the project, as it was a relatively new practice for most team members. Early on, tests were occasionally forgotten and written retrospectively, or not done properly due to inexperience. Over time, however, the team became more comfortable with writing tests along with their implementation.

Unit tests played an important role in catching bugs early in the code and verifying that each part of the code worked as expected on its own. In future projects, the team will try to adopt a habit of writing tests from the start, and invest more time in reviewing test quality alongside code quality.

Chapter 7

Conclusion

HeatItOn is a utility in the city of Heatington that supplies heat to approximately 1,600 buildings through a centralized district heating network. Prior to this project, heat production was planned manually, making it difficult to consistently select the most efficient combination of production units. The goal was to replace this manual process with an automated optimization system capable of generating heat production schedules that consistently meet heat demand while minimizing operational costs.

The system was built around five core modules: the Optimizer, Data Visualization, Asset Manager, Source Data Manager, and Result Data Manager. The architecture followed a clear separation between the Frontend, Backend, and Database layers.

Both required scenarios were successfully implemented. Scenario 1 covered a heat-only network, where the system prioritized cheaper units and handled maintenance scheduling. Scenario 2 introduced electricity production, where the optimizer calculated net production costs on an hourly basis to determine when to sell or consume electricity. A custom scenario was also implemented, giving users control over which assets are included in the optimization.

The final product successfully addressed the original problem statement. The system is able to automatically generate optimized heat production schedules that meet the heat demand of Heatington while minimizing operational costs, replacing the previous manual planning process entirely. Operators can now manage assets, import source data, run the optimizer, and view the results through a single integrated application.

Looking ahead, the system could be further extended in several ways. Integrating live electricity prices from Energinet.dk would allow the optimizer to react to real-time market data. The ability to enable or disable individual units directly from the user interface would give operators greater flexibility during optimization. In future development, the team would also focus on improving test coverage, refining the user interface based on user feedback, and expanding the system to support additional heating scenarios.

Chapter 8

AI Usage

Throughout the project, the team incorporated AI across multiple stages to assist with various tasks. Importantly, AI was never used as a replacement for human effort or core decision-making. Instead, it served exclusively as a supplementary tool to streamline team workflows and optimize overall time management.

Team had established a strict internal policy regarding AI assistance:

- **Sandboxed Interaction:** It was decided that AI models should be intentionally kept isolated from the development environment. Direct integration tools (such as IDE-embedded copilots that autonomously suggest or alter code) were prohibited. This ensured that no AI-generated code could interact directly with the active codebase.
- **Manual Verification and Rewriting:** Any logic, syntax, or formatting suggested by AI in external chats was carefully analyzed. No code was ever pasted directly into the project without proper human verification, testing, and manual rewriting to fit the specific architecture.
- **Total Accountability:** The team takes full responsibility for all final code, design choices, and documentation. AI was treated purely as an external advisory tool. Therefore, the team assumes complete ownership of the final implementation as if it were 100% manually produced, including any flaws or mistakes.
- **Original Authorship:** While AI assisted with debugging LaTeX [3] compilation errors and generating UML diagrams, all written content, structural logic, and academic analysis within this report are entirely the original work of the team members. No part of the report text was authored by AI.

Some of the examples using AI in this project:

- **Architectural Brainstorming:** In the early planning stages, AI was used to help generate ideas for potential design patterns and library integrations. The team then carefully weighed these options against established non-functional requirements before finalizing the architecture.
- **Debugging Assistance:** Dealing with undocumented Avalonia UI errors or sudden backend disconnects sometimes meant decoding vague error codes. AI was utilized as a

quick-reference resource to parse these codes, though diagnosing the root cause within our specific architecture and developing the actual fixes remained a manual team effort.

- **Formatting and Documentation:** To streamline team workflow, it was decided to outsource repetitive formatting tasks to AI tools. This included generating sketches and resolving some of tricky LaTeX compilation errors in the report, which saved valuable time without affecting the design or logic of the core system.
- **UI/UX Inspiration:** Various stylistic directions were explored for project logos and interface by generating mood boards with AI image tools. Even though the final application relies exclusively on original assets, these early visualizations helped steer overall design choices.
- **Code Review and Refactoring:** During the refactoring phase, AI acted as an extra set of eyes to catch missed references. When renaming methods within optimization modules, for instance, it helped verify that all old instances were properly updated, effectively reducing minor human errors without altering the underlying code logic.

AI models that were used by team members:

- Gemini-3.1 Pro
- Claude Opus/Sonnet
- Microsoft Copilot
- DeepSeek V3.2/V4
- ChatGPT
- GPT-5.3 Codex

Appendix A

UML Diagram

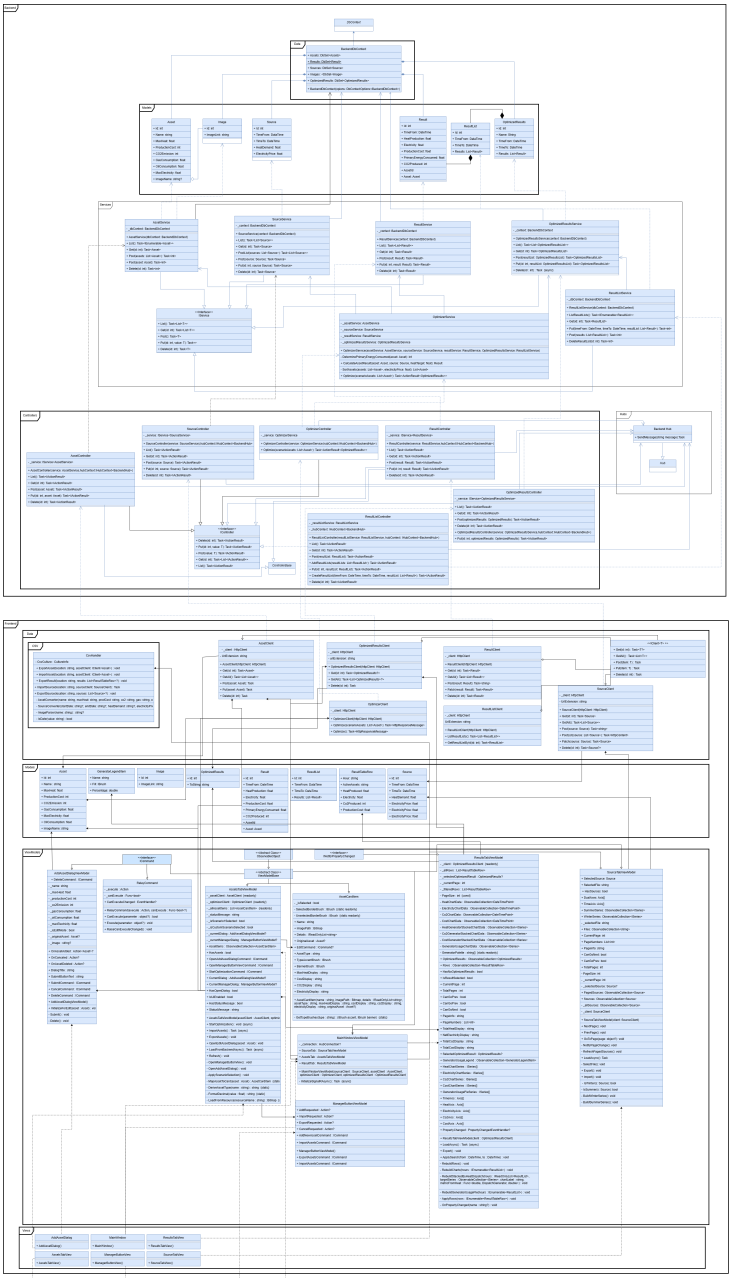


Figure A.1: Full frontend and backend detailed UML class diagram.

Appendix B

Task and Review Assignments

Table B.1: Code Review Assignments per Task

Issue Key	**Summary**	**Assignee**	**Review 1**	**Review 2**
HEAT-17	Problem statement	Adela Pastekova	Zoltan Suveges	Alex Zalanyi
HEAT-18	Problem analysis	Adela Pastekova	Filip Tichy	Melana Ohrimeca
HEAT-19	Make CRC cards	Filip Tichy	Zoltan Suveges	Marija Savkina
HEAT-20	Verb-Noun Analysis	Marija Savkina	Melana Ohrimeca	Alex Zalanyi
HEAT-21	Backend UML Diagram	Zoltan Suveges	Adela Pastekova	Filip Tichy
HEAT-22	Functional Requirements	Melana Ohrimeca	Marija Savkina	Adela Pastekova
HEAT-23	Write Non-Functional requirements	Melana Ohrimeca	Filip Tichy	Zoltan Suveges
HEAT-24	Write System requirements	Alex Zalanyi	Adela Pastekova	Melana Ohrimeca
HEAT-25	Write User Requirements	Alex Zalanyi	Marija Savkina	Zoltan Suveges
HEAT-26	Set up project	Zoltan Suveges	Everyone	—
HEAT-27	Create avalonia project	Zoltan Suveges	Everyone	—

Continued on next page

Table B.1: Code Review Assignments per Task (continued)

Issue Key	**Summary**	**Assignee**	**Review 1**	**Review 2**
HEAT-28	Create backend project	Zoltan Suveges	Everyone	—
HEAT-29	Write Dockerfile	Zoltan Suveges	Everyone	—
HEAT-30	Write docker-compose	Alex Zalanyi	Zoltan Suveges	—
HEAT-40	Dockerize backend and connect to dockerized database	Alex Zalanyi	Zoltan Suveges	—
HEAT-42	Connecting to database	Alex Zalanyi	Zoltan Suveges	—
HEAT-43	Create source data model	Alex Zalanyi	Zoltan Suveges	—
HEAT-77	Create asset data model	Alex Zalanyi	Zoltan Suveges	—
HEAT-78	The user should be able to list and sort source data	Zoltan Suveges	Filip Tichy	Marija Savkina
HEAT-81	Make UI Design	Everyone	—	—
HEAT-83	Frontend UML Diagram	Marija Savkina, Melana Ohrimeca	Zoltan Suveges	Alex Zalanyi
HEAT-91	The assets should be stored in the database	Marija Savkina	Melana Ohrimeca	Zoltan Suveges
HEAT-92	The source data should be stored in the database	Adela Pastekova	Zoltan Suveges	Filip Tichy
HEAT-93	Create result data model	Alex Zalanyi	Zoltan Suveges	—
HEAT-94	Read and write source (save/upload to/from a csv)	Adela Pastekova	Marija Savkina	Zoltan Suveges
HEAT-95	Read and write asset (save/upload to/from a csv)	Melana Ohrimeca	Alex Zalanyi	Filip Tichy

Continued on next page

Table B.1: Code Review Assignments per Task (continued)

Issue Key	**Summary**	**Assignee**	**Review 1**	**Review 2**
HEAT-98	Make UI for managing assets (add, edit and delete)	Marija Savkina	Melana Ohrimeca	Zoltan Suveges
HEAT-101	Make UI to set up and start scenarios	Marija Savkina	Zoltan Suveges	Adela Pastekova
HEAT-103	Make UI to list previous results	Filip Tichy	Alex Zalanyi	Zoltan Suveges
HEAT-104	Make UI to show selected result	Filip Tichy	Melana Ohrimeca	Adela Pastekova
HEAT-119	Implement CRUD operations for source data	Adela Pastekova	Filip Tichy	Zoltan Suveges
HEAT-122	Create template for source item	Zoltan Suveges	Filip Tichy	Marija Savkina
HEAT-123	Create UI to list source data	Zoltan Suveges	Melana Ohrimeca	Adela Pastekova
HEAT-124	Rename demand to source	Alex Zalanyi	—	—
HEAT-129	Assets view design	Everyone	—	—
HEAT-130	Source data view design	Everyone	—	—
HEAT-131	Optimize view design	Everyone	—	—
HEAT-132	Upload view	Everyone	—	—
HEAT-133	Result data view design	Everyone	—	—
HEAT-147	Create result end points	Melana Ohrimeca	Adela Pastekova	Alex Zalanyi
HEAT-148	Create resultList end points	Filip Tichy	Zoltan Suveges	Alex Zalanyi
HEAT-149	Create UI architecture and tabs with empty pages	Adela Pastekova	Melana Ohrimeca	Filip Tichy

Continued on next page

Table B.1: Code Review Assignments per Task (continued)

Issue Key	**Summary**	**Assignee**	**Review 1**	**Review 2**
HEAT-150	Create API connector to UI	Zoltan Suveges	Alex Zalanyi	Melana Ohrimeca
HEAT-151	Unit test for asset manager	Marija Savkina	Zoltan Suveges	Adela Pastekova
HEAT-152	Unit test for source manager	Adela Pastekova	Marija Savkina	Zoltan Suveges
HEAT-153	Create controller (optimizer end points)	Alex Zalanyi	Zoltan Suveges	Filip Tichy
HEAT-154	Create service (optimizer end points)	Alex Zalanyi	Melana Ohrimeca	Marija Savkina
HEAT-155	Unit test (optimizer end points)	Alex Zalanyi	Adela Pastekova	Marija Savkina
HEAT-156	Create algorithm for optimization	Alex Zalanyi, Filip Tichy	Alex Zalanyi	—
HEAT-157	The user should be able to see images for each generator	Zoltan Suveges	Adela Pastekova	Alex Zalanyi
HEAT-167	Make template layout for asset card	Marija Savkina	Adela Pastekova	Filip Tichy
HEAT-168	Get data from backend (assets)	Marija Savkina	Zoltan Suveges	Filip Tichy
HEAT-169	Write result (save/upload to/from a csv)	Melana Ohrimeca	Alex Zalanyi	Zoltan Suveges
HEAT-170	Make charts for result data	Zoltan Suveges	Adela Pastekova	Alex Zalanyi
HEAT-171	Have chart filter options	Filip Tichy	Melana Ohrimeca	Alex Zalanyi
HEAT-172	Connect to backend	Filip Tichy	Alex Zalanyi	Marija Savkina
HEAT-176	Create asset csv	Alex Zalanyi	—	—

Continued on next page

Table B.1: Code Review Assignments per Task (continued)

Issue Key	**Summary**	**Assignee**	**Review 1**	**Review 2**
HEAT-178	Chart on source view only refreshes when resizing	Zoltan Suveges	—	—
HEAT-183	Redo Asset Manager buttons	Marija Savkina	Adela Pastekova	Zoltan Suveges
HEAT-188	Rework result data	Zoltan Suveges	Adela Pastekova	Alex Zalanyi
HEAT-198	Interface for controllers	Filip Tichy	Marija Savkina	Zoltan Suveges
HEAT-199	Interface for services	Adela Pastekova	Filip Tichy	Melana Ohrimeca
HEAT-200	Interface for HttpClients	Melana Ohrimeca	Adela Pastekova	Zoltan Suveges
HEAT-201	Combine CSV handlers into one class	Zoltan Suveges	Filip Tichy	Melana Ohrimeca
HEAT-203	Rework charts and diagrams	Marija Savkina	Filip Tichy	Adela Pastekova
HEAT-207	Nicer UI	Adela Pastekova	Melana Ohrimeca	Zoltan Suveges
HEAT-208	Auto refresh	Filip Tichy	Adela Pastekova	Marija Savkina
HEAT-209	Centralized backend error handling with custom errors	Zoltan Suveges	Filip Tichy	Adela Pastekova
HEAT-233	Edit and delete source data from the source tab	Filip Tichy	Marija Savkina	Alex Zalanyi
HEAT-237	Delete saved optimization results	Adela Pastekova	Filip Tichy	Melana Ohrimeca
HEAT-242	Electricity price API feasibility (Energinet.dk)	Alex Zalanyi	Filip Tichy	Zoltan Suveges
HEAT-278	Update sources not reupload them	Zoltan Suveges	Filip Tichy	Adela Pastekova

Continued on next page

Table B.1: Code Review Assignments per Task (continued)

Issue Key	**Summary**	**Assignee**	**Review 1**	**Review 2**
HEAT-279	Graph on source view not showing	Adela Pastekova	Zoltan Suveges	Marija Savkina

Appendix C

Report Table of Contribution

Table C.1: Report Writing Contributions

Chapter	**Author**	**First Review**	**Second Review**
AI Usage	Melana Ohrimeca	Alex Zalanyi	Marija Savkina
Abstract	Marija Savkina	Zoltan Suveges	Adela Pastekova
Appendix	Adela Pastekova	Marija Savkina	Filip Tichy
Conclusion	Adela Pastekova	Alex Zalanyi	Melana Ohrimeca
Discussion and Reflection	Adela Pastekova	Marija Savkina	Melana Ohrimeca
Frontend Implementation, Midterm Version (5.1.1) (Implementation and Technical Realization)	Alex Zalanyi	Melana Ohrimeca	Adela Pastekova
Final Version, Midterm Version (5.2.1), Midterm Version (5.3.1) (Implementation and Technical Realization)	Zoltan Suveges	Alex Zalanyi	Adela Pastekova
Final Version (5.2.2) Final Version (5.3.2) (Implementation and Technical Realization)	Marija Savkina	Zoltan Suveges	Melana Ohrimeca
Project Objectives and Requirements (Introduction)	Marija Savkina	Filip Tichy	Melana Ohrimeca

Continued on next page

Table C.1: Report Writing Contributions (continued)

Chapter	**Author**	**First Review**	**Second Review**
Problem Statement (Introduction)	Adela Pastekova	Zoltan Suveges	Alex Zalanyi
Problem Analysis (Introduction)	Adela Pastekova	Filip Tichy	Melana Ohrimeca
Functional Requirements (Introduction)	Melana Ohrimeca	Marija Savkina	Adela Pastekova
Non-Functional Requirements (Introduction)	Melana Ohrimeca	Filip Tichy	Zoltan Suveges
System Requirements (Introduction)	Alex Zalanyi	Adela Pastekova	Melana Ohrimeca
Target User Persona Requirements (Appendix)	Alex Zalanyi	Marija Savkina	Zoltan Suveges
References	Melana Ohrimeca	Marija Savkina	Alex Zalanyi
Scrum Implementation	Filip Tichy	Melana Ohrimeca	Adela Pastekova
Verb-noun analysis (Solution (Technical Aspect))	Marija Savkina	Melana Ohrimeca	Alex Zalanyi
CRC Cards (Solution (Technical Aspect))	Filip Tichy	Zoltan Suveges	Marija Savkina
System Design – Class UML Diagram (Solution (Technical Aspect))	Melana Ohrimeca	Alex Zalanyi	Marija Savkina
System Design – Class Sequence Diagram (Solution (Technical Aspect))	Marija Savkina	Alex Zalanyi	Melana Ohrimeca
System Technical Architecture, Package Diagram, Activity Diagrams (Solution (Technical Aspect))	Alex Zalanyi	Adela Pastekova	Melana Ohrimeca
Sprints	Filip Tichy	Alex Zalanyi	Melana Ohrimeca

Table C.2: Project Class Contribution

Project	**Folder**	**File**	**Author**
Backend	Data	BackendDbContext.cs	Zoltan Suveges
	Controllers	AssetsController.cs	Marija Savkina
		OptimizedResultsController.cs	Zoltan Suveges
		OptimizerController.cs	Alex Zalanyi
		ResultByHourController.cs	Filip Tichy
		ResultController.cs	Melana Ohrimeca
		SourceController.cs	Adela Pastekova
	Hubs	BackendHubs.cs	Zoltan Suveges
	Models	Asset.cs	Alex Zalanyi
		OptimizedResult.cs	Zoltan Suveges
		Result.cs	Alex Zalanyi
		ResultByHour.cs	Zoltan Suveges
		Source.cs	Alex Zalanyi
	Interfaces	IController.cs	Filip Tichy
		IService.cs	Adela Pastekova
	Services	AssetService.cs	Marija Savkina
		OptimizedResultsService.cs	Zoltan Suveges
		OptimizerService.cs	Alex Zalanyi
		ResultByHourService.cs	Filip Tichy
		ResultService.cs	Melana Ohrimeca
		SourceService.cs	Adela Pastekova
Frontend	Data	CsvHandler.cs	Adela Pastekova & Melana Ohrimeca
		AssetClient.cs	Zoltan Suveges
		IClient.cs	Zoltan Suveges
		OptimizedResultsClient.cs	Zoltan Suveges
		OptimizerClient.cs	Zoltan Suveges
		ResultClient.cs	Zoltan Suveges
		ResultListClient.cs	Zoltan Suveges

Continued on next page

Table C.2: Project Class Contribution (continued)

Project	**Folder**	**File**	**Author**
Frontend	Data	SourceClient.cs	Zoltan Suveges
	Models	Asset.cs	Zoltan Suveges
		GeneratorLegendItem.cs	Marija Savkina
		OptimizedResults.cs	Zoltan Suveges
		Result.cs	Zoltan Suveges
		ResultList.cs	Zoltan Suveges
		ResultTableRow.cs	Zoltan Suveges
		Source.cs	Zoltan Suveges
	ViewModels	AddAssetDialogViewModel.cs	Marija Savkina
		AssetsTabViewModel.cs	Marija Savkina
		MainWindowViewModel.cs	Everyone
		ManagerButtonViewModel.cs	Marija Savkina
		ResultsTabViewModel.cs	Adela Pastekova & Marija Savkina & Alex Zalanyi
		SourceTabViewModel.cs	Zoltan Suveges & Melana Ohrimeca & Adela Pastekova
		ViewModelBase.cs	Zoltan Suveges
	Views	AddAssetDialog.axaml	Marija Savkina & Adela Pastekova
		AssetsTabView.axaml	Marija Savkina & Adela Pastekova
		MainWindow.axaml	Everyone
		ManagerButtonView.axaml	Marija Savkina & Adela Pastekova
		ResultsTabView.axaml	Marija Savkina & Alex Zalanyi & Adela Pastekova
		SourceTabView.axaml	Zoltan Suveges & Filip Tichy & Melana Ohrimeca

Appendix D

Figures

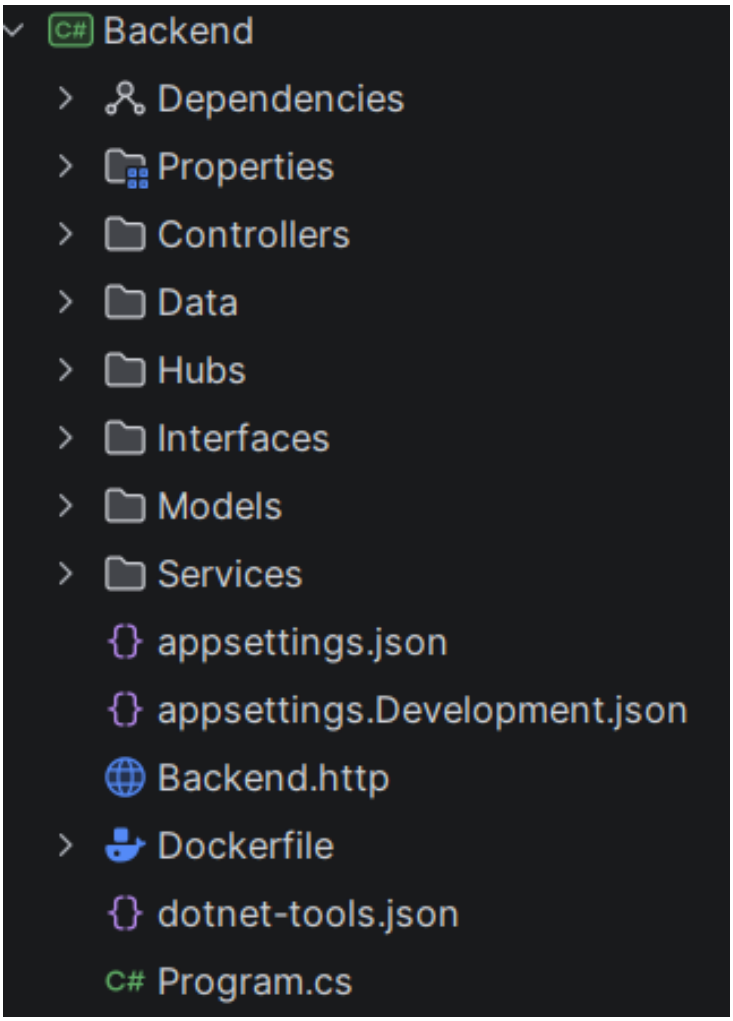


Figure D.1: Backend file structure showing the organization of controllers, services, interfaces, models, and data components.

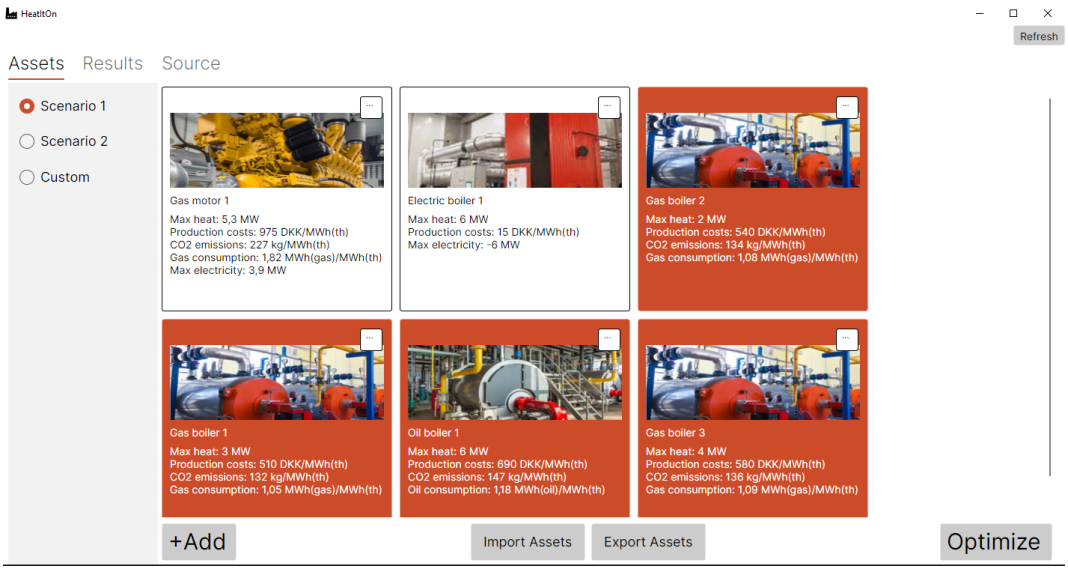


Figure D.2: Assets tab from the demo version

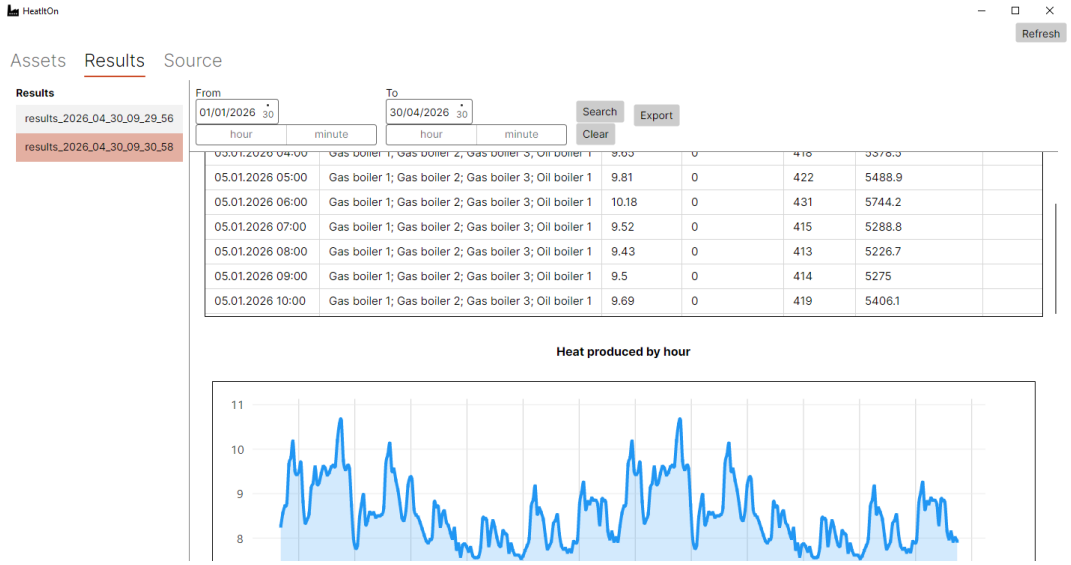


Figure D.3: Results tab from the demo version

Sprint 1

Sprint 1 Planning:

HEAT-21 | Task |Backend UML Diagram done by Zoltan Suveges(Left in under review from prev. sprint as last-minute changes were made).

HEAT-81 | Task | Make UI Design done by Everyone together

HEAT-23 | Task | Write Non-Functional requirements done by Melana Ohrimeca

HEAT-24 | Task | Write System Requirements done by Alex Zalanyi

HEAT-91| Task | The assets should be stored in the database done by Marija Savkina

HEAT-83| Task | Frontend UML Diagram done by Marija Savkina and Melana Ohrimeca

HEAT-89| Bug | EF error done by Filip Tichy

HEAT-125| Bug | EF error done by Filip Tichy

HEAT-92| Task | The source data should be stored in the DB done by Adela Pastekova

HEAT-25| Task | Write user requirements done by Alex Zalanyi

Figure D.4: Sprint 1 planning

~~HEAT-92~~ The source data should be stored in the database	Adela Pašteková	Zoltán Süveges	= Medium	DONE
~~HEAT-91~~ The assets should be stored in the database	Marija Savkina	Zoltán Süveges	= Medium	DONE
~~HEAT-83~~ Frontend UML Diagram	Marija Savkina	Zoltán Süveges	= Medium	DONE
~~HEAT-81~~ Make Ui Design	Everyone	Zoltán Süveges	= Medium	DONE
~~HEAT-25~~ Write User Requirements	Alex Zalanyi	Zoltán Süveges	= Medium	DONE
~~HEAT-24~~ Write System requirements	Alex Zalanyi	Zoltán Süveges	= Medium	DONE
~~HEAT-23~~ Write Non-Functional requirements	Melana Ohrimeca	Zoltán Süveges	= Medium	DONE
~~HEAT-21~~ Backend UML Diagram	Zoltán Süveges	Zoltán Süveges	= Medium	DONE

Figure D.5: Sprint 1 backlog

	▶️ Start doing	🔴 Stop doing	🙌 Keep doing
1	• distribute tasks properly according to everyone's own skills	assign a task just so everyone has something to do ignoring the difficulty or if we even have to do it now	• Planning the meetings ahead with clear agenda and time
2	do user story estimates properly	don't move prescheduled meetings last minute just because we have blank classes	communicating
3	bringing cake if you r late😊👌😍💞😊👍😍❤️🌕🍩👊🍰	assigning multiple hard and not completely understood tasks to one person no matter if they are almost the same	reviews for every task
4	retrospective review if we changed what we said we would		If someone doesnt have a knowledge of something it is always being explain by more pro teammates😄👍
5	start focusing more during meetings so we don't spend unnecessary time		notes from every meeting
6	sprint planning on fridays		Assigning hard tasks to several people
7			Amazing discussions

Figure D.6: Sprint 1 retrospective

Sprint 2

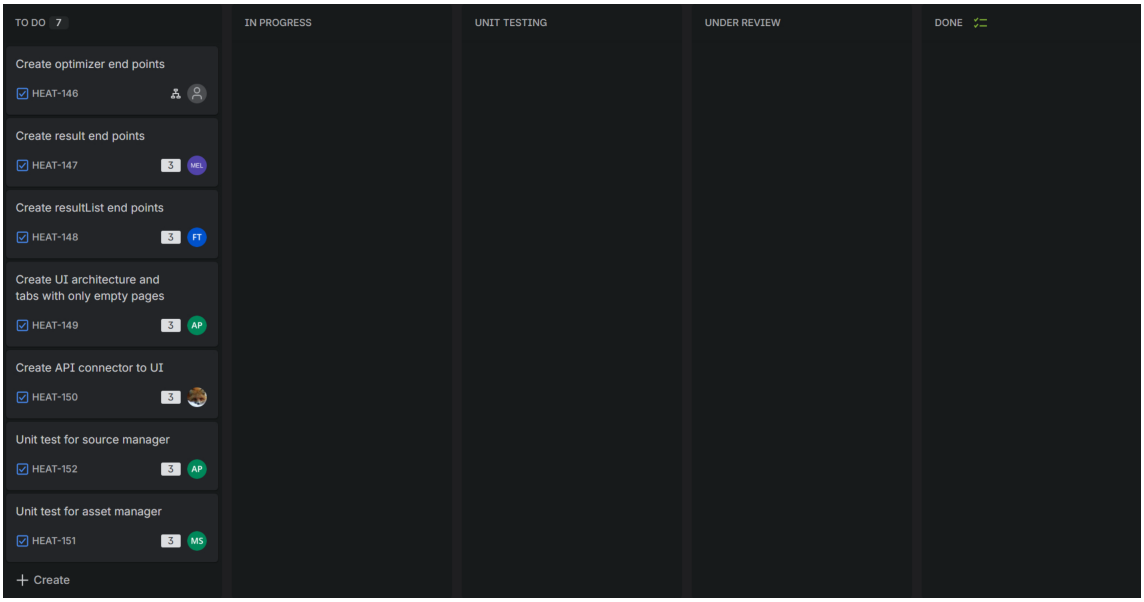


Figure D.7: Sprint 2 planning

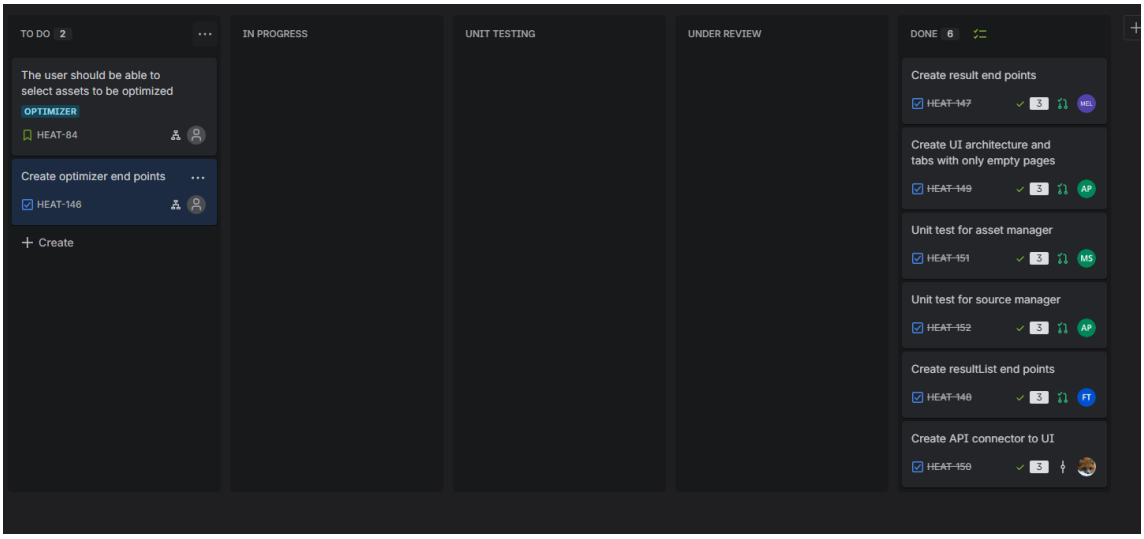


Figure D.8: Sprint 2 backlog

▶️ Start doing	🛑 Stop doing	🙌 Keep doing
• Start having a proper plan what do to every meeting so we wont waste time		• Planning meetings ahead
Runing your code before you start a PR	putting 2 person on related tasks in one sprint	Documenting retro, reviews and daily stand ups
First writing unit test before moving it to waiting for the approval		reaching out for help and helping each other when in need.
announce noshows(even for softeng)		win hackathons
soft deadline for tasks on wednesday evening for review thursday meeting		distribute tasks properly according to everyone's own skills
take reviews seriously		do user story estimates properly
connecting tasks have half sprints		retrospective review if we changed what we said we would
if you dont understand task fully you ask for another person to be asign as well		

Figure D.9: Sprint 2 retrospective

Sprint 3

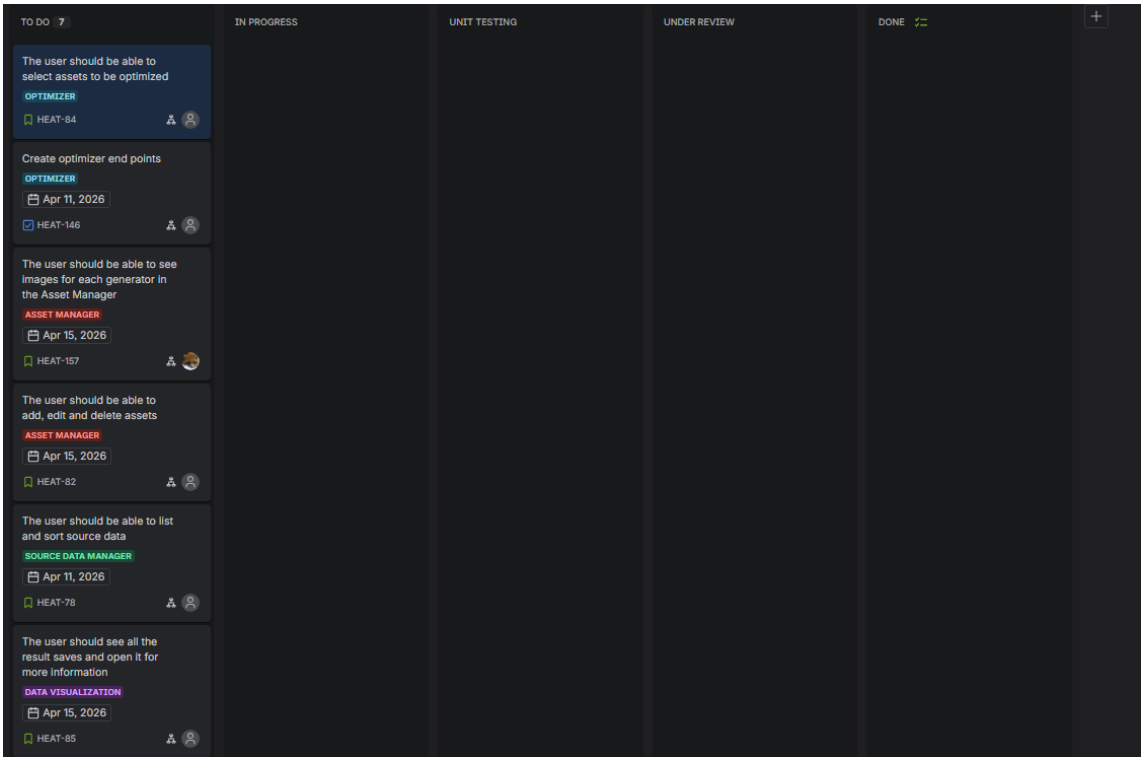


Figure D.10: Sprint 3 planning (1)

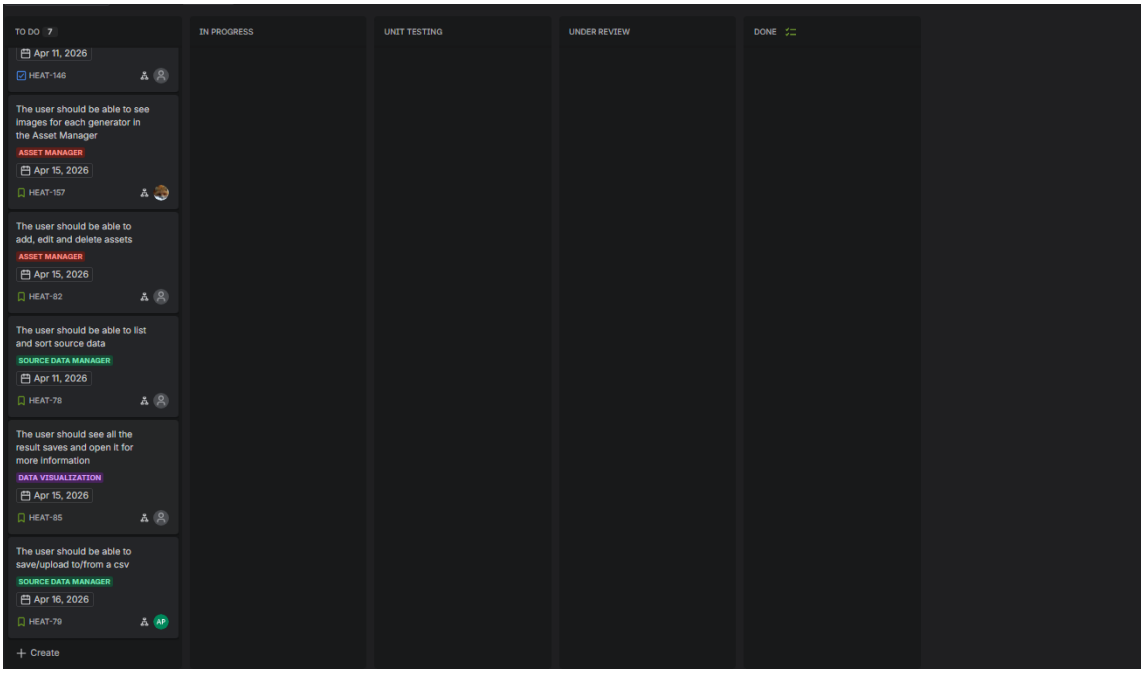


Figure D.11: Sprint 3 planning (2)

~~HEAT-176~~ Create asset csv	Alex Zalanyi	Alex Zalanyi	= Medium	DONE
~~HEAT-146~~ Create optimizer end points	Alex Zalanyi	Zoltán Süveges	= Medium	DONE
~~HEAT-85~~ The user should see all the result saves and ope...	Filip Tichý	Zoltán Süveges	= Medium	DONE
~~HEAT-84~~ The user should be able to select assets to be o...	Marija Savkina	Zoltán Süveges	= Medium	DONE
~~HEAT-82~~ The user should be able to add, edit and delete ...	Marija Savkina	Zoltán Süveges	= Medium	DONE
~~HEAT-79~~ The user should be able to save/upload to/from ...	Adela Pašteková	Alex Zalanyi	= Medium	DONE
~~HEAT-78~~ The user should be able to list and sort source d...	Zoltán Süveges	Alex Zalanyi	= Medium	DONE

Figure D.12: Sprint 3 backlog

▶ Start doing	🛑 Stop doing	🙌 Keep doing
• when leaving comments in Jira or github, dm or mention that in discord in case it goes unseen	• more than 2 review for one branch	• bringing candies if arrive late
Ask for help if you don't understand something		Start having a proper plan what do to every meeting so we wont waste time
Telling in advance if you have something to do and don't have as much time for the project		Running your code before you start a PR
announce noshows(even for softeng)		First writing unit test before moving it to waiting for the approval
when assigned to work with someone do work together		soft deadline for tasks on wednesday evening for review thursday meeting
deadlines		take reviews seriously
follow deadlines		keep doing reviews table

Figure D.13: Sprint 3 retrospective

Sprint 4

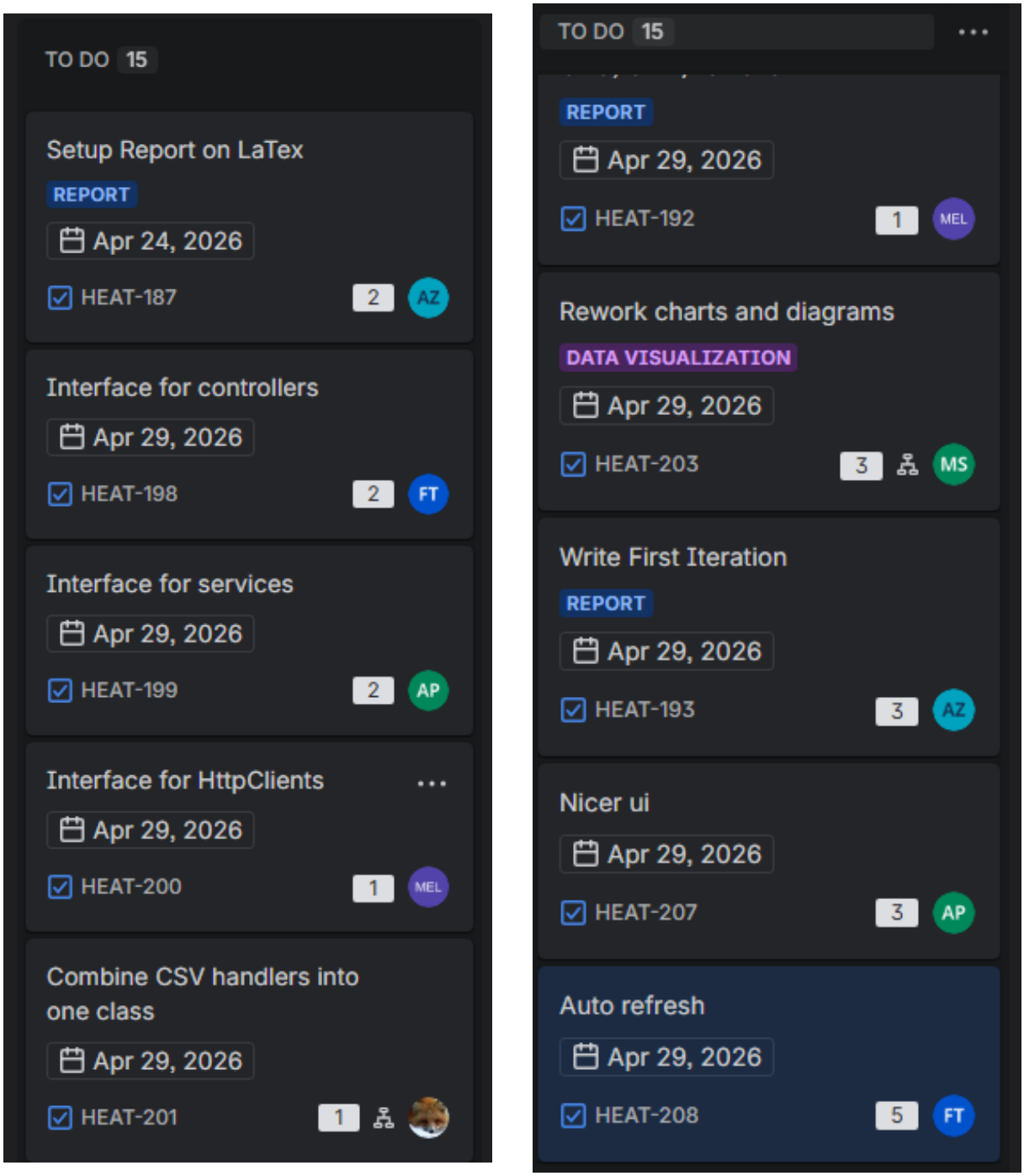


Figure D.14: Sprint 4 planning

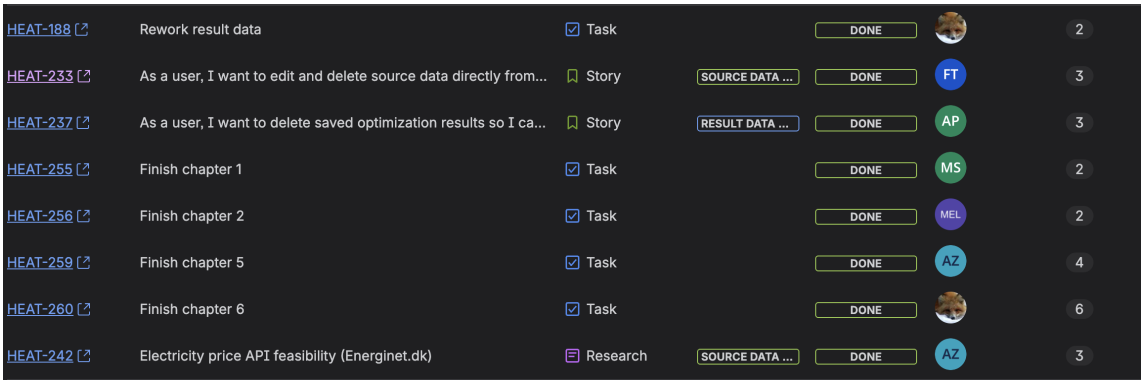
~~HEAT-208~~ Auto refresh	Filip Tichý	Zoltán Süveges	= Medium	DONE
~~HEAT-207~~ Nicer ui	Adela Pašteková	Zoltán Süveges	= Medium	DONE
~~HEAT-203~~ Rework charts and diagrams	Marija Savkina	Marija Savkina	= Medium	DONE
~~HEAT-201~~ Combine CSV handlers into one class	Zoltán Süveges	Zoltán Süveges	= Medium	DONE
~~HEAT-200~~ Interface for HttpClients	Melana Ohrimeca	Zoltán Süveges	= Medium	DONE
~~HEAT-199~~ Interface for services	Adela Pašteková	Zoltán Süveges	= Medium	DONE
~~HEAT-198~~ Interface for controllers	Filip Tichý	Zoltán Süveges	= Medium	DONE
~~HEAT-193~~ Write First Iteration	Alex Zalanyi	Alex Zalanyi	= Medium	DONE
~~HEAT-192~~ Move Planning(Verbnoun, CRC, UML) to LaTex	Melana Ohrimeca	Alex Zalanyi	= Medium	DONE
~~HEAT-191~~ Move Requirements to LaTex	Melana Ohrimeca	Alex Zalanyi	= Medium	DONE
~~HEAT-190~~ Move Problem Analysis to LaTex	Melana Ohrimeca	Alex Zalanyi	= Medium	DONE
~~HEAT-189~~ Move Problem Statement to LaTex	Filip Tichý	Alex Zalanyi	= Medium	DONE
~~HEAT-187~~ Setup Report on LaTex	Alex Zalanyi	Alex Zalanyi	= Medium	DONE
~~HEAT-183~~ Redo Asset Manager Buttons	Marija Savkina	Alex Zalanyi	= Medium	DONE

Figure D.15: Sprint 4 backlog

▶️ Start doing	🛑 Stop doing	🙌 Keep doing
• Finish the tasks on time	• crying 😞	• not have more than 2 reviews per branch
• paying attention to explanations	• move meeting time 15 min before meeting xd	• bringing candies if arrive late
• doing reviews on time		• Start having a proper plan what do to every meeting so we wont waste time
• taking deadlines seriously		• Ask for help if you don't understand something

Figure D.16: Sprint 4 retrospective

Sprint 5



HEAT-188	Rework result data	Task		DONE		2
HEAT-233	As a user, I want to edit and delete source data directly from...	Story	SOURCE DATA ...	DONE	FT	3
HEAT-237	As a user, I want to delete saved optimization results so I ca...	Story	RESULT DATA ...	DONE	AP	3
HEAT-255	Finish chapter 1	Task		DONE	MS	2
HEAT-256	Finish chapter 2	Task		DONE	MEL	2
HEAT-259	Finish chapter 5	Task		DONE	AZ	4
HEAT-260	Finish chapter 6	Task		DONE		6
HEAT-242	Electricity price API feasibility (Energinet.dk)	Research	SOURCE DATA ...	DONE	AZ	3

Figure D.17: Sprint 5 backlog

▶ Start doing	🛑 Stop doing	🙌 Keep doing
• taking mental walks	being passive aggresive	• bringing candy if you are late
touch grass	UMLs	deadlines
	being late 🫠	
	leave in the middle of project month	

Figure D.18: Sprint 5 retrospective

Sprint 6

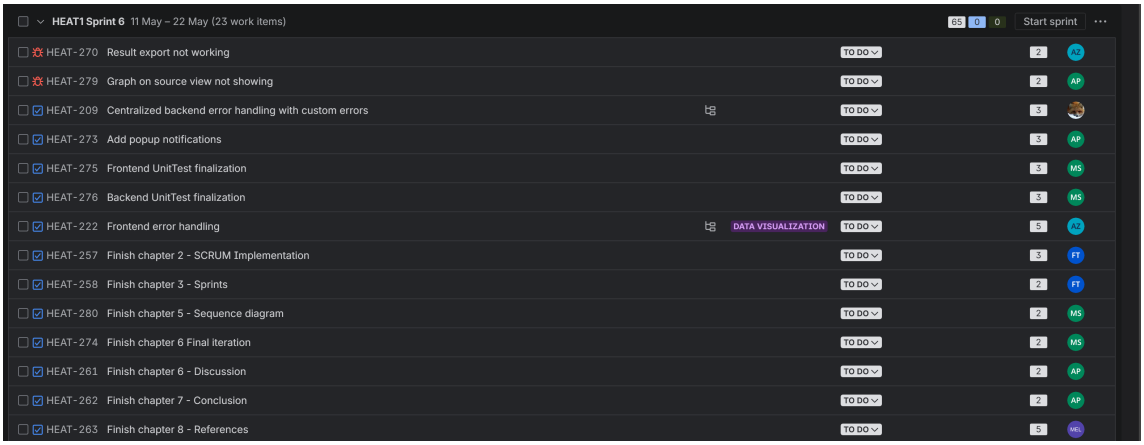


Figure D.19: Sprint 6 planning (1)

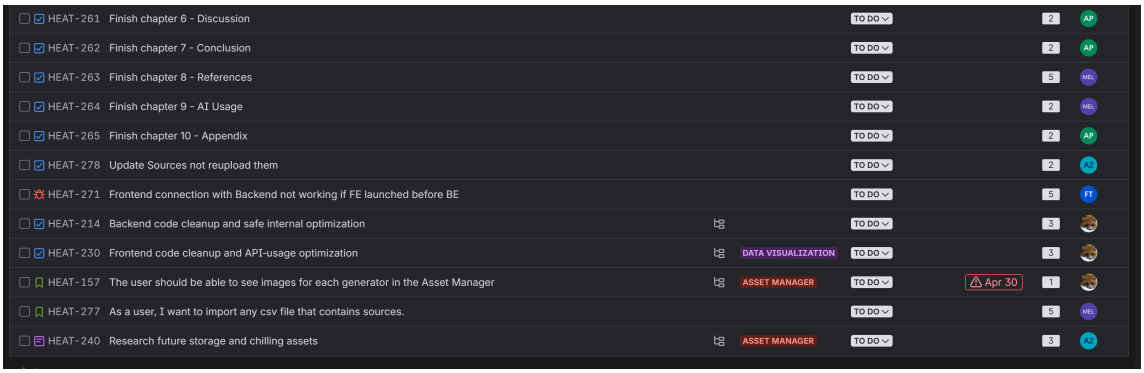


Figure D.20: Sprint 6 planning (2)

▶️ Start doing	🛑 Stop doing	🙌 Keep doing
• doing reviews on time	• bad disagreements handling	• more wonderful projects like this😊
study for exams	coming late	finishing code on time
enjoy summer holidays	mixing worklife and bringing outside negative energy to group	

Figure D.21: Sprint 6 retrospective

Appendix E

Target User Persona

The system is designed with a specific target audience in mind. To ensure an optimal user experience, the ideal user (persona) interacting with this system is assumed to have the following profile:

- Be able to read in English
- Be able to navigate a simple UI
- Have a basic understanding of the district heating problem being solved by the system
- Be able to read and understand basic input/output data

Appendix F

References

[1] Avalonia UI. *Avalonia UI - Cross Platform .NET UI Framework*. 2026. URL: https://avaloniaui.net/.

[2] Paul DuBois. *MySQL*. Addison-Wesley, 2013.

[3] Helmut Kopka and Patrick W Daly. *Guide to LATEX*. Pearson Education, 2003.

[4] Henrik Lund et al. "4th Generation District Heating (4GDH): Integrating smart thermal grids into future sustainable energy systems". In: *Energy* 68 (2014), pp. 1–11.

[5] Rob Miles. *C# Programming: The Yellow Book*. 2019. URL: https://www.scribd.com/document/750548686/C-Yellow-Book.

[6] Ken Schwaber and Jeff Sutherland. *The Scrum Guide*. 2020. URL: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf.

[7] University of Southern Denmark (SDU). *Semester Project Data: Asset consumption and producion*. Internal document provided via Itslearning. 2026.

[8] University of Southern Denmark (SDU). *Semester Project Data: Heat Demand and Electricity Prices*. Internal dataset provided via Itslearning. 2026.